



UNIVERSIDAD NACIONAL DE INGENIERÍA

**ÁREA DE CONOCIMIENTO DE LA TECNOLOGIA DE LA
INFORMACIÓN Y COMUNICACIÓN.**

Ingeniería de Sistemas

Programación II

(DACTIC)

“Trabajo de los 10 ejercicios en C#”

Autores:

- Br. Cristóbal Darío Díaz Mora 2024-1568U
- Br. Pablo Adonis Mendieta Ocampo 2024-1592U
- Br. Adriadna Julisa González Reyes 2024-1571U
- Br. Wilbort Paul Rivera Ocón 2024-1416U

Grupo:

2T1-SIS-S

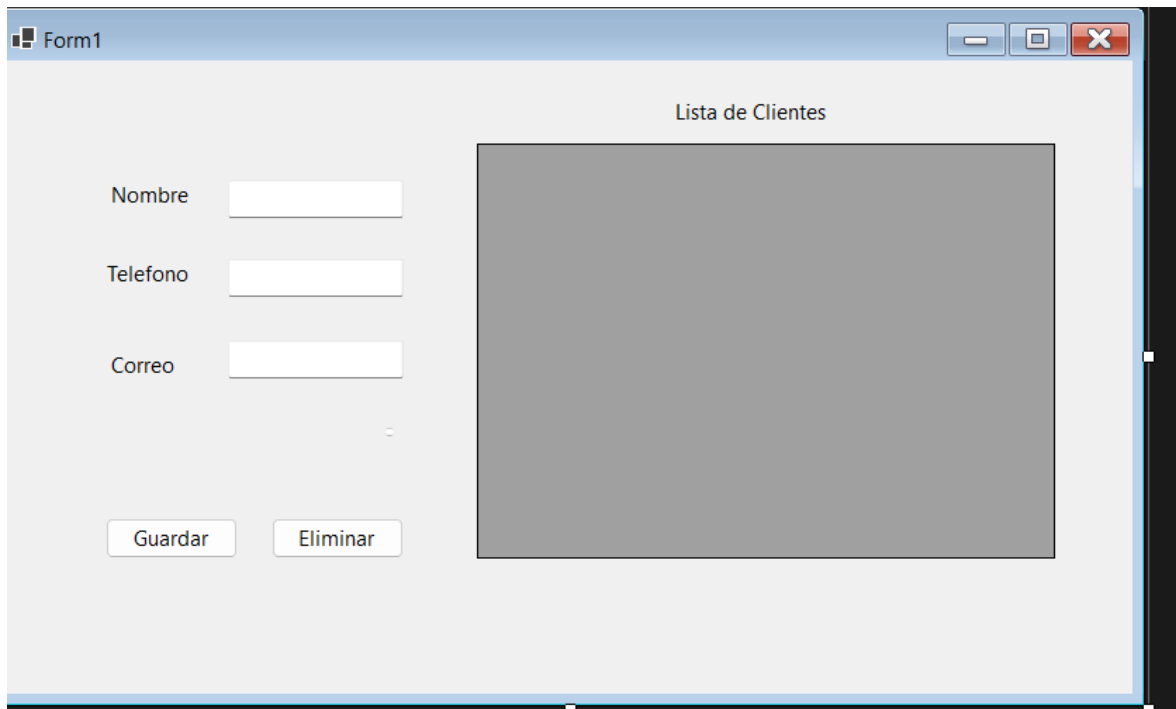
Docente:

Doc. Abel Edmundo Marín Reyes.

12 de mayo de 2026. Managua, Nicaragua.

Ejercicio 1

Este es el ejercicio número 1 el diseño del sistema de empleados está conformado con los botones respectivos, dgv, los label, los txt, etc



The image shows a screenshot of a Windows Forms application window titled "Form1". The window contains a form titled "Lista de Clientes". On the left side, there are three text boxes labeled "Nombre", "Telefono", and "Correo". Below these text boxes are two buttons labeled "Guardar" and "Eliminar". On the right side, there is a large gray rectangular area representing a data grid (DataGridView).

El diseño del sistema de gestión de empleados en Windows Forms, detallando la interfaz visual con botones, etiquetas y cuadros de texto para capturar datos de clientes. Se presenta la configuración técnica de la clase ConexionDB que establece la comunicación con el servidor local de SQL Server para la base de datos ClientesDB

La conexión a la base datos de la computadora

```
1 using System;
2 using System.Collections.Generic;
3 using System.Data.SqlClient;
4 using System.Linq;
5 using System.Text;
6 using System.Threading.Tasks;
7
8 namespace ejercicio1
9 {
10     2 referencias
11     public class ConexionDB
12     {
13         private string cadena =
14             "Server=localhost;Database=ClientesDB;Trusted_Connection=True;TrustServerCertificate=True;";
15
16         3 referencias
17         public SqlConnection ObtenerConexion()
18         {
19             return new SqlConnection(cadena);
20         }
21     }
22 }
```

El ejemplo se enfoca en la arquitectura del proyecto mediante la implementación del patrón de diseño Repository en la clase ClienteRepository. Esta página detalla el código C# necesario para la operación de "Agregar" registros, utilizando bloques using para asegurar el cierre de conexiones y parámetros SQL para proteger el sistema contra ataques de inyección

El repositorio en general en base a todos los datos.

```
1 using System;
2 using System.Collections.Generic;
3 using System.Data.SqlClient;
4 using System.Linq;
5 using System.Text;
6 using System.Threading.Tasks;
7 using System.Windows.Forms;
8
9 namespace ejercicio1
10 {
11     1 referencia
12     public class ClienteRepository : IClienteRepository
13     {
14         private ConexionDB conexion = new ConexionDB();
15
16         2 referencias
17         public void Agregar(Cliente cliente)
18         {
19             using (SqlConnection conn = conexion.ObtenerConexion())
20             {
21                 string query = "INSERT INTO Clientes (Nombre, Telefono, Correo) VALUES (@Nombre, @Telefono, @Correo)";
22                 SqlCommand cmd = new SqlCommand(query, conn);
23                 cmd.Parameters.AddWithValue("@Nombre", cliente.Nombre);
24                 cmd.Parameters.AddWithValue("@Telefono", cliente.Telefono);
25                 cmd.Parameters.AddWithValue("@Correo", cliente.Correo);
26
27                 try
28                 {
29                     conn.Open();
30                     cmd.ExecuteNonQuery();
31                 }
32                 catch (Exception ex)
33                 {
34                     MessageBox.Show("Error al guardar: " + ex.Message);
35                 }
36             }
37         }
38     }
39
40     2 referencias
41     public List<Cliente> ObtenerTodos()
42     {
43         List<Cliente> lista = new List<Cliente>();
44
45         using (SqlConnection conn = conexion.ObtenerConexion())
46         {
47             string query = "SELECT * FROM Clientes";
48             SqlCommand cmd = new SqlCommand(query, conn);
49
50             try
51             {
52                 conn.Open();
53                 SqlDataReader reader = cmd.ExecuteReader();
54
55                 while (reader.Read())
56                 {
57                     Cliente cliente = new Cliente
58                     {
59                         Id = Convert.ToInt32(reader["Id"]),
60                         Nombre = reader["Nombre"].ToString(),
61                         Telefono = reader["Telefono"].ToString(),
62                         Correo = reader["Correo"].ToString()
63                     };
64                     lista.Add(cliente);
65                 }
66             }
67             catch (Exception ex)
68             {
69                 MessageBox.Show("Error al cargar datos: " + ex.Message);
70             }
71         }
72     }
73 }
```

Implementación de los get y set a cada asignación dada.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace ejercicio1
8  {
9      9 referencias
10     public class Cliente
11     {
12         1 referencia
13         public int Id { get; set; }
14
15         3 referencias
16         public string Nombre { get; set; }
17
18         3 referencias
19         public string Telefono { get; set; }
20
21         3 referencias
22         public string Correo { get; set; }
23     }
24 }
```

Se describe el modelo de datos de la clase Cliente utilizando propiedades auto-implementadas con métodos get y set para manejar los atributos de Id, Nombre, Telefono y Correo. Además, muestra la configuración del punto de entrada de la aplicación en Program.cs, encargada de inicializar el formulario principal del Sistema

```

74         return lista;
75     }
76
77     2 referencias
78     public void Eliminar(int id)
79     {
80         using (SqlConnection conn = conexion.ObtenerConexion())
81         {
82             string query = "DELETE FROM Clientes WHERE Id = @Id";
83             SqlCommand cmd = new SqlCommand(query, conn);
84             cmd.Parameters.AddWithValue("@Id", id);
85
86             try
87             {
88                 conn.Open();
89                 cmd.ExecuteNonQuery();
90             }
91             catch (Exception ex)
92             {
93                 MessageBox.Show("Error al eliminar: " + ex.Message);
94             }
95         }
96     }
97 }
98
99
100
101

```

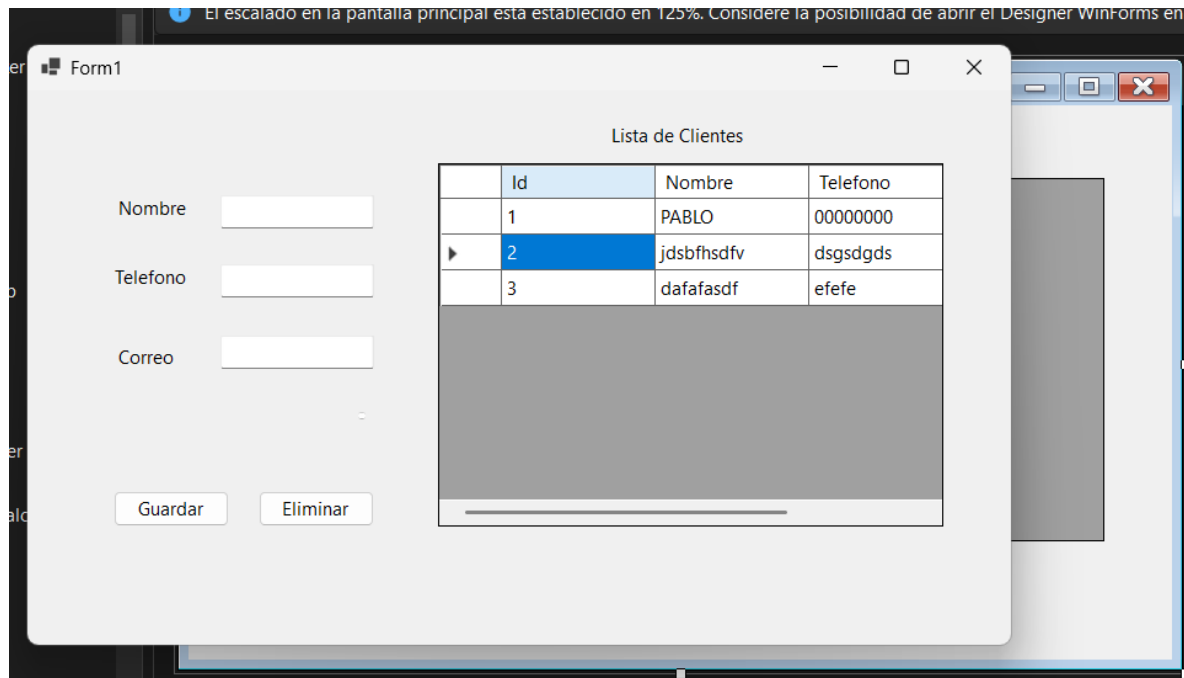
La imagen muestra específicamente el método **Eliminar** dentro de la clase ClienteRepository. Este código se encarga de borrar un registro de la base de datos utilizando el Id del cliente como referencia. Para hacerlo, abre una conexión a SQL Server y ejecuta la instrucción DELETE FROM Clientes WHERE Id = @Id, empleando un parámetro para asegurar que solo se elimine el registro seleccionado y proteger la base de datos de errores o comandos malintencionados.

```

namespace ejercicio1
{
    0 referencias
    internal static class Program
    {
        [STAThread]
        0 referencias
        static void Main()
        {
            https://aka.ms/applicationconfiguration.
            ApplicationConfiguration.Initialize();
            Application.Run(new Form1());
        }
    }
}

```

Este bloque prepara la configuración básica y abre mi formulario principal



Esta imagen muestra la implementación del método **Agregar** (o Insertar) dentro de la clase **ClienteRepository**. El código utiliza una sentencia SQL **INSERT INTO** para registrar un nuevo cliente en la tabla, pasando los valores de **Nombre**, **Telefono** y **Correo** como parámetros. Es fundamental notar el uso del bloque **using** (`var command = new SqlCommand(...)`), lo que asegura que los recursos se liberen correctamente una vez que el registro se guarda en la base de datos, manteniendo la eficiencia del sistema.

LA base de datos es la siguiente, la cual nos sirve para guardar todos los campos del código:

```
CREATE DATABASE ClientesDB;
```

```
GO
```

```
USE ClientesDB;
```

```
GO
```

```
CREATE TABLE dbo.Clientes (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Nombre NVARCHAR(100) NOT NULL,
    Telefono NVARCHAR(20) NOT NULL,
    Correo NVARCHAR(100) NOT NULL
);
```

```
GO
```

Todo el programa corriendo y rectificando que está en la base de datos.

```
1 SELECT * FROM Clientes;
```

100 %  No se encontraron problemas.

Resultados Mensajes

	Id	Nombre	Telefono	Correo
1	1	PABLO	00000000	CARADETURKA@GMAIL.COM
2	2	jdsbfhsdfv	dsgsdgds	sgddsgdsg
3	3	dafafasdf	efefe	fefefe

Esta es una consulta básica que se hace con un Select From para evaluar que la base de datos esta conectada correctamente al proyecto en visual estudio, en donde se muestra evidencia que se están registrando datos a la base de datos

Ejercicio 2

Sistema de Producción

Esta guía explica paso a paso cómo se desarrolló el sistema de producción utilizando Windows Forms y SQL Server para gestionar productos, inventario y registros.

Paso 1: Creación del Proyecto

Se creó un proyecto Windows Forms App (.NET Framework) en Visual Studio y se asignó el nombre correspondiente al sistema de producción.

Paso 2: Creación de la Base de Datos

En SQL Server Management Studio se creó la base de datos para almacenar productos y movimientos de inventario.

```
CREATE DATABASE SistemaProduccion;  
GO  
USE SistemaProduccion;
```

```
CREATE TABLE Productos(  
    Id INT PRIMARY KEY IDENTITY(1,1),  
    Nombre NVARCHAR(100),  
    Cantidad INT,  
    Precio DECIMAL(10,2)  
);
```

Paso 4: Conexión a la Base de Datos

Se implementó una clase de conexión para permitir la comunicación con SQL Server.

```
namespace SistemaProductos  
{  
    using System.Data.SqlClient;  
  
    // references  
    public class Conexion  
    {  
        private string cadena = "Server=localhost;Database=adri;Trusted_Connection=True;";  
  
        // references  
        public SqlConnection GetConnection()  
        {  
            return new SqlConnection(cadena);  
        }  
    }  
}
```

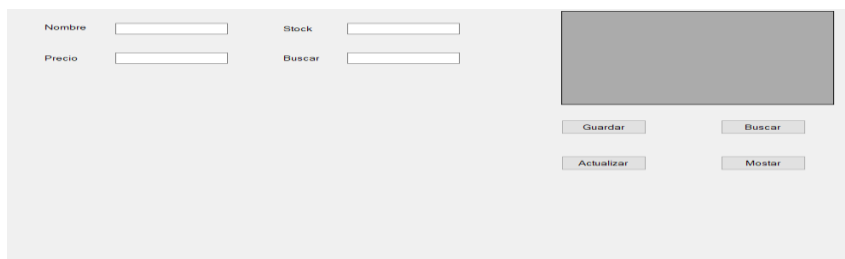
Paso 5: Implementación del Repository

Se utilizó el patrón Repository para separar la lógica de acceso a datos de la interfaz gráfica.

```
namespace SistemaProductos
{
    0 references
    public class Producto
    {
        0 references
        public int Id { get; set; }
        0 references
        public string Nombre { get; set; }
        0 references
        public decimal Precio { get; set; }
        0 references
        public int Stock { get; set; }
    }
}
```

Paso 6: Diseño del Formulario

En el formulario principal se agregaron controles para registrar y visualizar productos.



TextBox -> txtNombre

TextBox -> txtCantidad

TextBox -> txtPrecio

Button -> btnGuardar

Button -> btnEliminar

DataGridView -> dgvProductos

Paso 7: Guardar los Productos

Se programó el botón Guardar para registrar productos en la base de datos.

```
1 reference
private void btnGuardar_Click(object sender, EventArgs e)
{
    conexion.Open();

    string query = "INSERT INTO Productos (Nombre, Precio, Stock) VALUES (@nombre, @precio, @stock)";
    SqlCommand cmd = new SqlCommand(query, conexion);

    cmd.Parameters.AddWithValue("@nombre", txtNombre.Text);
    cmd.Parameters.AddWithValue("@precio", decimal.Parse(txtPrecio.Text));
    cmd.Parameters.AddWithValue("@stock", int.Parse(txtStock.Text));

    cmd.ExecuteNonQuery();
    conexion.Close();

    MessageBox.Show("Producto guardado");
}
```

Paso 8: Visualización de Datos

Los productos registrados se muestran automáticamente en un DataGridView.

```
private void btnMostrar_Click(object sender, EventArgs e)
{
    conexion.Open();

    SqlDataAdapter da = new SqlDataAdapter("SELECT * FROM Productos", conexion);
    DataTable dt = new DataTable();

    da.Fill(dt);
    dgvProductos.DataSource = dt;

    conexion.Close();
}
```

Paso 9: Buscar Producto

El sistema permite llevar control de cantidades disponibles y productos registrados.

```
private void btnBuscar_Click(object sender, EventArgs e)
{
    conexion.Open();

    string query = "SELECT * FROM Productos WHERE Nombre LIKE @nombre";
    SqlDataAdapter da = new SqlDataAdapter(query, conexion);

    da.SelectCommand.Parameters.AddWithValue("@nombre", "%" + txtBuscar.Text + "%");

    DataTable dt = new DataTable();
    da.Fill(dt);

    dgvProductos.DataSource = dt;

    conexion.Close();
}
```

Paso 10: Actualizar Producto

El sistema permite actualizar las cantidades disponibles y productos registrados agrgados por el cliente en el sistema.

```
private void btnActualizar_Click(object sender, EventArgs e)
{
    conexion.Open();

    int id = Convert.ToInt32(dgvProductos.CurrentRow.Cells["Id"].Value);

    string query = "UPDATE Productos SET Nombre=@nombre, Precio=@precio, Stock=@stock WHERE Id=@id";
    SqlCommand cmd = new SqlCommand(query, conexion);

    cmd.Parameters.AddWithValue("@nombre", txtNombre.Text);
    cmd.Parameters.AddWithValue("@precio", decimal.Parse(txtPrecio.Text));
    cmd.Parameters.AddWithValue("@stock", int.Parse(txtStock.Text));
    cmd.Parameters.AddWithValue("@id", id);

    cmd.ExecuteNonQuery();
    conexion.Close();

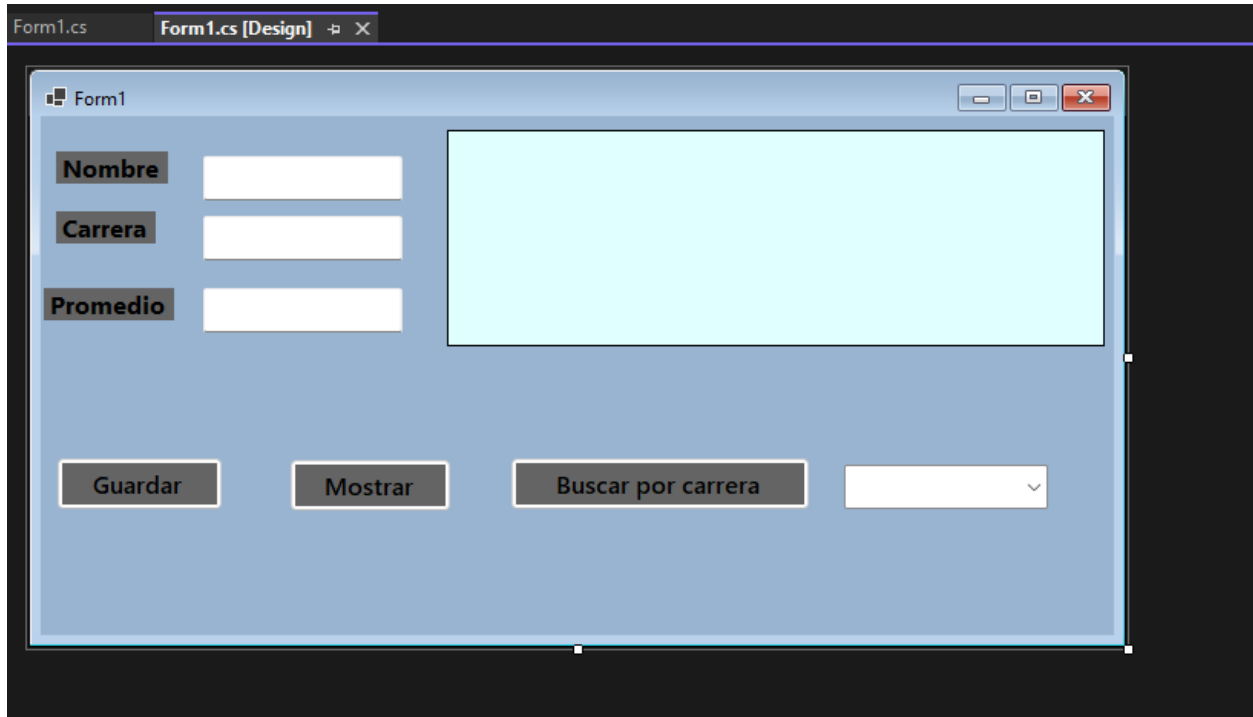
    MessageBox.Show("Producto actualizado");
}
```

Conclusión

El sistema de producción desarrollado permite registrar y administrar productos e inventario utilizando Windows Forms y SQL Server, aplicando buenas prácticas de organización y separación de responsabilidades

Ejercicio 3

DOCUMENTACION DE 3CER EJERCICIO



Campo "Nombre"

- Tipo: TextBox
- Función: Permite ingresar el nombre del estudiante.

Campo "Carrera"

- Tipo: TextBox
- Función: Permite ingresar la carrera del estudiante.

Campo "Promedio"

- Tipo: TextBox
- Función: Permite ingresar el promedio académico del estudiante.

Tabla de Datos

- Tipo: DataGridView
- Función: Muestra los registros almacenados de los estudiantes.

Botón "Guardar"

- Tipo: Button
- Función: Guarda la información ingresada en el sistema.

Botón "Mostrar"

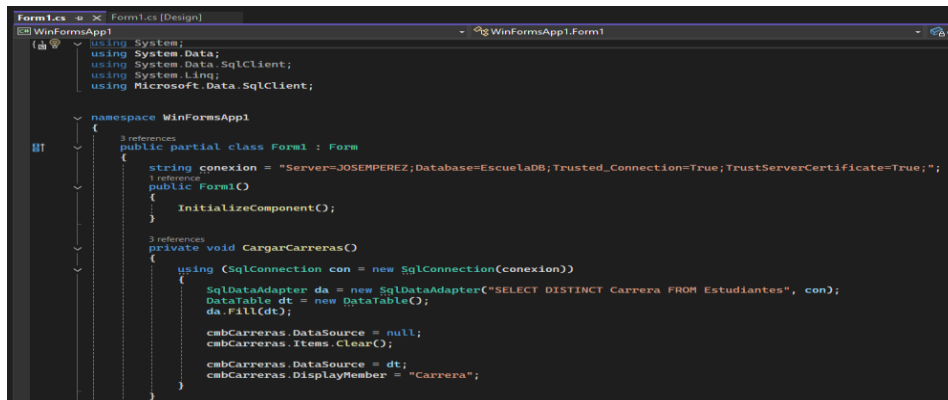
- Tipo: Button
- Función: Muestra todos los registros almacenados.

Botón "Buscar por carrera"

- Tipo: Button
- Función: Realiza búsquedas de estudiantes según la carrera seleccionada.

Lista Desplegable

- Tipo: ComboBox
- Función: Permite seleccionar una carrera para realizar la búsqueda.



```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Linq;
using Microsoft.Data.SqlClient;

namespace WinFormsApp1
{
    public partial class Form1 : Form
    {
        string conexion = "Server=JOSEMPEREZ;Database=EscuelaDB;Trusted_Connection=True;TrustServerCertificate=True;";

        public Form1()
        {
            InitializeComponent();
        }

        private void CargarCarreras()
        {
            using (SqlConnection con = new SqlConnection(conexion))
            {
                SqlDataAdapter da = new SqlDataAdapter("SELECT DISTINCT Carrera FROM Estudiantes", con);
                DataTable dt = new DataTable();
                da.Fill(dt);

                cmbCarreras.DataSource = null;
                cmbCarreras.Items.Clear();

                cmbCarreras.DataSource = dt;
                cmbCarreras.DisplayMember = "Carrera";
            }
        }
    }
}
```

Configuración de la Conexión: Se define una cadena llamada conexion que contiene la dirección del servidor (JOSEMPEREZ) y el nombre de la base de datos (EscuelaDB).

Creación del Método CargarCarreras(): Se inicia un bloque using para abrir una conexión segura con la base de datos. El uso de using asegura que la conexión se cierre automáticamente al terminar para no desperdiciar recursos.

Consulta de Datos (SQL): Se prepara un "adaptador" (SqlDataAdapter) con la instrucción SQL: SELECT DISTINCT Carrera FROM Estudiantes

Busca todos los nombres de las carreras en la tabla "Estudiantes", eliminando los duplicados (DISTINCT).

Almacenamiento Temporal: Se crea un objeto llamado DataTable dt. Este funciona como una tabla virtual en la memoria de la computadora donde se guardan los resultados que trae el adaptador de la base de datos.

Limpieza del Control: Antes de mostrar los nuevos datos, se limpia el ComboBox (cmbCarreras.DataSource = null y .Items.Clear()) para evitar errores o que se duplique la información si se vuelve a llamar al método.

Vinculación (Binding): Finalmente, se asigna la tabla de datos al control:

- **DataSource:** Se le dice al ComboBox que sus datos vienen de dt.
- **DisplayMember:** Se le indica que el texto que debe mostrar al usuario es el de la columna "Carrera".

```
private void btnGuardar_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(txtNombre.Text) ||
        string.IsNullOrEmpty(txtCarrera.Text) ||
        string.IsNullOrEmpty(txtPromedio.Text))
    {
        MessageBox.Show("Todos los campos deben estar llenos");
        return;
    }

    if (!decimal.TryParse(txtPromedio.Text, out decimal promedio))
    {
        MessageBox.Show("El promedio debe ser un número válido");
        return;
    }

    using (SqlConnection con = new SqlConnection(conexion))
    {
        string query = "INSERT INTO Estudiantes (Nombre, Carrera, Promedio) VALUES (@nombre, @carrera, @promedio)";

        SqlCommand cmd = new SqlCommand(query, con);
        cmd.Parameters.AddWithValue("@nombre", txtNombre.Text);
        cmd.Parameters.AddWithValue("@carrera", txtCarrera.Text);
        cmd.Parameters.AddWithValue("@promedio", promedio);

        con.Open();
        cmd.ExecuteNonQuery();
        con.Close();

        MessageBox.Show("Estudiante guardado");
    }
}
```

Validación de Campos Vacíos: El programa revisa si los cuadros de texto de Nombre, Carrera o Promedio están vacíos o solo tienen espacios. Si falta algún dato, muestra un mensaje de advertencia y detiene el proceso con un return.

Validación de Formato Numérico: Intenta convertir el texto del campo Promedio a un número decimal (decimal.TryParse). Si el usuario escribió letras o un formato inválido, muestra un error y detiene la ejecución.

Preparación de la Sentencia SQL: Se crea una variable query con la instrucción INSERT INTO. Se utilizan **parámetros** (como @nombre) en lugar de escribir los valores directamente. Esto es una medida de seguridad fundamental para evitar ataques de inyección SQL.

Asignación de Valores: Mediante cmd.Parameters.AddWithValue, se vincula el contenido de los cuadros de texto del formulario con los parámetros de la consulta SQL definidos en el paso anterior.

Ejecución de la Operación:

- **con.Open():** Abre la comunicación con el servidor de base de datos.
- **cmd.ExecuteNonQuery():** Ejecuta la instrucción de inserción. Se usa "NonQuery" porque es una operación que cambia datos pero no devuelve una tabla de resultados.
- **con.Close():** Cierra la conexión inmediatamente para liberar recursos.

Confirmación al Usuario: Si todo sale bien, el programa muestra un cuadro de mensaje (MessageBox) indicando que el "Estudiante guardado" con éxito.

```
1 reference
private void btnBuscar_Click(object sender, EventArgs e)
{
    using (SqlConnection con = new SqlConnection(conexion))
    {
        SqlDataAdapter da = new SqlDataAdapter("SELECT * FROM Estudiantes", con);
        DataTable dt = new DataTable();
        da.Fill(dt);

        var filtrados = dt.AsEnumerable()
            .Where(x => x.Field<string>("Carrera") == cmbCarreras.Text);

        if (filtrados.Any())
            dgvEstudiantes.DataSource = filtrados.CopyToDataTable();
        else
            MessageBox.Show("No hay resultados");
    }
}

1 reference
private void btnMostrar_Click(object sender, EventArgs e)
{
    using (SqlConnection con = new SqlConnection(conexion))
    {
        SqlDataAdapter da = new SqlDataAdapter("SELECT * FROM Estudiantes", con);
        DataTable dt = new DataTable();
        da.Fill(dt);

        var ordenados = dt.AsEnumerable()
            .OrderByDescending(x => x.Field<decimal>("Promedio"));

        dgvEstudiantes.DataSource = ordenados.CopyToDataTable();
    }
}
```

Método btnBuscar_Click (Filtrado de datos)

Este bloque de código se encarga de buscar estudiantes que pertenezcan a una carrera específica seleccionada por el usuario.

- **Carga inicial:** Se conecta a la base de datos y trae todos los registros de la tabla "Estudiantes" a una tabla en memoria (DataTable dt).

- Filtrado con LINQ: Utiliza una consulta lógica para separar solo las filas donde la columna "Carrera" coincida exactamente con el texto seleccionado en el ComboBox (cmbCarreras).
- Validación y Muestra:
 - Si encuentra resultados (.Any()), convierte esos datos filtrados en una nueva tabla y la muestra en la cuadrícula de la pantalla (dgvEstudiantes).
 - Si no hay coincidencias, muestra un mensaje avisando que "No hay resultados".

2. Método btnMostrar_Click (Ordenamiento de datos)

Este método se utiliza para visualizar a todos los estudiantes, pero organizados por sus calificaciones.

- Obtención de datos: Al igual que el anterior, abre la conexión y descarga la información completa de los estudiantes en un DataTable.
- Ordenamiento Lógico: En lugar de filtrar, utiliza la instrucción .OrderByDescending.

```
1 reference
private void Form1_Load(object sender, EventArgs e)
{
    CargarCarreras();

    CargarCarreras();

    using (SqlConnection con = new SqlConnection(conexion))
    {
        SqlDataAdapter da = new SqlDataAdapter("SELECT DISTINCT Carrera FROM Estudiantes", con);
        DataTable dt = new DataTable();
        da.Fill(dt);

        cmbCarreras.DataSource = dt;
        cmbCarreras.DisplayMember = "Carrera";
    }
}
```

Llamada a Métodos Externos: El código comienza llamando dos veces a la función CargarCarreras()..

1. Apertura de Conexión: Se establece un bloque using con una nueva SqlConnection. Esto abre el canal de comunicación con la base de datos de manera segura y eficiente.
2. Consulta de Carreras Únicas: Se utiliza un SqlDataAdapter para ejecutar la instrucción: SELECT DISTINCT Carrera FROM Estudiantes
 - ¿Qué hace?: Busca en la base de datos todos los nombres de las carreras disponibles, pero sin repetir ninguno (DISTINCT).

3. Llenado de Tabla en Memoria: Los datos obtenidos se guardan en un DataTable dt. Este objeto actúa como un contenedor temporal de información dentro del código.
4. Configuración del ComboBox (cmbCarreras):
 - o DataSource: Se vincula el control de la pantalla con la tabla de datos (dt).
 - o DisplayMember: Se especifica que el texto que el usuario leerá en la lista desplegable es el nombre de la columna "Carrera".

```
1
2 CREATE DATABASE EscuelaDB;
3 GO
4
5 USE EscuelaDB;
6 GO
7
8 --ejecutas para crear la tabla
9
10 CREATE TABLE Estudiantes (
11     Id INT IDENTITY(1,1) PRIMARY KEY,
12     Nombre NVARCHAR(100),
13     Carrera NVARCHAR(100),
14     Promedio DECIMAL(5,2)
15 );
16
--
```

Por últimos tenemos la base de datos cual ponemos los campos que queremos guardar como id, nombre, carrera y promedio una vez que ponemos los campos que irán en la base de datos conectaremos nuestra base de datos con el código mediante la clase conexión.

Ejercicio 4

Sistema de empleados

Sistema de empleados
Registro, edición y consultas sobre la nómina

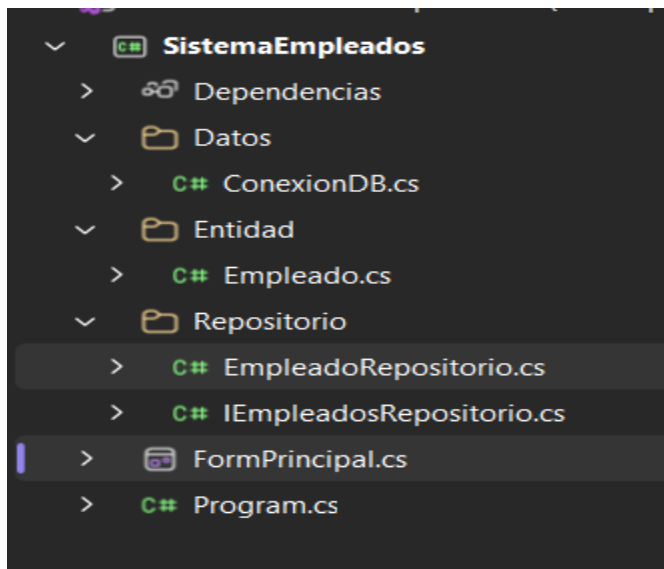
Nombre Cargo Salario

Agregar Actualizar Eliminar

Total de salarios Empleado con mayor

```
Panel panelEncabezado;  
TableLayoutPanel tableLayoutEncabezado;  
Label lblTituloEncabezado;  
Label lblSubtituloEncabezado;  
Panel panelCuerpo;  
Panel panelTarjetaDatos;  
TableLayoutPanel tableLayoutTarjeta;  
TableLayoutPanel tableLayoutEntrada;  
Panel panelFilaSeparador;  
Panel separadorAcciones;  
FlowLayoutPanel flowAccionesCrud;  
FlowLayoutPanel flowAccionesInformes;  
Panel panelGridMarco;  
Panel panelGridInterior;  
DataGridView dgvEmpleados;  
Label lblNombre;  
TextBox txtNombre;  
Label lblCargo;  
TextBox txtCargo;  
Label lblSalario;  
TextBox txtSalario;  
Button btnAgregar;  
Button btnActualizar;  
Button btnEliminar;  
Button btnTotalSalarios;  
Button btnEmpleadoMayorSalario;
```

La interfaz gráfica está conformada de todo lo mencionado anteriormente como 7 panel, 10 label entre las otras etc.



De esta manera tenemos estructurado nuestro código el cual explicaremos a continuación.

Carpeta datos, clase Conexion

```
using Microsoft.Data.SqlClient;

namespace SistemaEmpleados.Datos;

4 referencias
public static class ConexionDB
{
    private const string CadenaConexion =
        @"Server=LAPTOP-QI6QQ6L5;Database=SistemaEmpleados;Integrated Security=True;TrustServerCertificate=True;";

    4 referencias
    public static SqlConnection ObtenerConexion()
    {
        return new SqlConnection(CadenaConexion);
    }
}
```

Este código define una clase estática llamada **ConexionDB** que centraliza la configuración de acceso a la base de datos del sistema. En ella se guarda una cadena de conexión con los parámetros necesarios para conectarse a SQL Server (nombre del servidor, base de datos, seguridad integrada y certificado confiable). A través del método público y estático **ObtenerConexion()**, se devuelve un objeto **SqlConnection** listo para usarse en cualquier parte del programa, lo que permite abrir una conexión a la base de datos de manera sencilla y reutilizable sin tener que repetir la configuración en cada archivo.

Carpeta entidad, clase Empleado

```
namespace SistemaEmpleados.Entidad;

16 referencias
public class Empleado
{
    5 referencias
    public int Id { get; set; }
    7 referencias
    public string Nombre { get; set; } = string.Empty;
    6 referencias
    public string Cargo { get; set; } = string.Empty;
    9 referencias
    public decimal Salario { get; set; }
}
```

Este código define la clase **Empleado**, que funciona como una entidad o modelo dentro del sistema. Su propósito es representar la información básica de un empleado en la aplicación. La clase contiene propiedades como **Id** (identificador único), **Nombre** (texto con valor inicial vacío), **Cargo** (puesto de trabajo, también inicializado como cadena vacía) y **Salario** (valor decimal). En conjunto, esta estructura permite almacenar y manipular los datos de cada empleado de manera ordenada y coherente, sirviendo como puente entre la base de datos y la lógica del programa.

Carpeta repositorio, clase empleado repositorio, iempleadorepositorio.

```
namespace SistemaEmpleados.Repositorio;

1 referencia
public class EmpleadoRepositorio : IEmpleadosRepositorio
{
    4 referencias
    public IReadOnlyList<Empleado> ObtenerTodos()
    {
        const string sql = "SELECT Id, Nombre, Cargo, Salario FROM Empleados ORDER BY Id;";
        var lista = new List<Empleado>();

        using var conexion = ConexionDB.ObtenerConexion();
        conexion.Open();
        using var comando = new SqlCommand(sql, conexion);
        using var lector = comando.ExecuteReader();
        while (lector.Read())
        {
            lista.Add(new Empleado
            {
                Id = lector.GetInt32(0),
                Nombre = lector.GetString(1),
                Cargo = lector.GetString(2),
                Salario = lector.GetDecimal(3)
            });
        }

        return lista;
    }

    2 referencias
    public int Insertar(Empleado empleado)
    {
        const string sql = """
            INSERT INTO Empleados (Nombre, Cargo, Salario)
            VALUES (@Nombre, @Cargo, @Salario);
            SELECT CAST(SCOPE_IDENTITY() AS int);
            """;

        using var conexion = ConexionDB.ObtenerConexion();
        conexion.Open();
        using var comando = new SqlCommand(sql, conexion);
        comando.Parameters.AddWithValue("@Nombre", empleado.Nombre);
        comando.Parameters.AddWithValue("@Cargo", empleado.Cargo);
        comando.Parameters.AddWithValue("@Salario", empleado.Salario);

        var resultado = comando.ExecuteScalar();
        return resultado is int id ? id : Convert.ToInt32(resultado);
    }
}
```

```

        using var conexion = ConexionDB.ObtenerConexion();
        conexion.Open();
        using var comando = new SqlCommand(sql, conexion);
        comando.Parameters.AddWithValue("@Nombre", empleado.Nombre);
        comando.Parameters.AddWithValue("@Cargo", empleado.Cargo);
        comando.Parameters.AddWithValue("@Salario", empleado.Salario);

        var resultado = comando.ExecuteScalar();
        return resultado is int id ? id : Convert.ToInt32(resultado);
    }

    2 referencias
    public int Actualizar(Empleado empleado)
    {
        const string sql = """
            UPDATE Empleados
            SET Nombre = @Nombre, Cargo = @Cargo, Salario = @Salario
            WHERE Id = @Id;
            """;

        using var conexion = ConexionDB.ObtenerConexion();
        conexion.Open();
        using var comando = new SqlCommand(sql, conexion);
        comando.Parameters.AddWithValue("@Id", empleado.Id);
        comando.Parameters.AddWithValue("@Nombre", empleado.Nombre);
        comando.Parameters.AddWithValue("@Cargo", empleado.Cargo);
        comando.Parameters.AddWithValue("@Salario", empleado.Salario);

        return comando.ExecuteNonQuery();
    }

    2 referencias
    public int Eliminar(int id)
    {
        const string sql = "DELETE FROM Empleados WHERE Id = @Id;";

        using var conexion = ConexionDB.ObtenerConexion();
        conexion.Open();
        using var comando = new SqlCommand(sql, conexion);
        comando.Parameters.AddWithValue("@Id", id);

        return comando.ExecuteNonQuery();
    }

    2 referencias
    public decimal TotalSalarios()
    {
        var empleados = ObtenerTodos();

```

```

        return comando.ExecuteNonQuery();
    }

    2 referencias
    public int Eliminar(int id)
    {
        const string sql = "DELETE FROM Empleados WHERE Id = @Id;";

        using var conexion = ConexionDB.ObtenerConexion();
        conexion.Open();
        using var comando = new SqlCommand(sql, conexion);
        comando.Parameters.AddWithValue("@Id", id);

        return comando.ExecuteNonQuery();
    }

    2 referencias
    public decimal TotalSalarios()
    {
        var empleados = ObtenerTodos();
        return empleados.Sum(e => e.Salario);
    }

    public Empleado? SalarioMasAlto()
    {
        var empleados = ObtenerTodos();
        if (empleados.Count == 0)
        {
            return null;
        }

        var salarioMaximo = empleados.Max(e => e.Salario);
        return empleados.FirstOrDefault(e => e.Salario == salarioMaximo);
    }

```

La clase `EmpleadoRepositorio` se encarga de gestionar la información de los empleados en la base de datos utilizando C# y `Microsoft.Data.SqlClient`. Implementa la interfaz `IEmployadosRepositorio`, por lo que define métodos para realizar operaciones CRUD: obtener empleados, insertar nuevos registros, actualizar información y eliminar empleados. Cada método abre una conexión con la base de datos mediante `ConexionDB.ObtenerConexion()` y ejecuta consultas SQL usando `SqlCommand`. Además, se utilizan parámetros (`@Nombre`, `@Cargo`, etc.) para enviar datos de forma segura y evitar problemas como SQL Injection.

El método `ObtenerTodos()` consulta todos los empleados y convierte cada fila en un objeto `Empleado` usando `SqlDataReader`. `Insertar()` agrega un nuevo empleado y devuelve el ID generado automáticamente, mientras que `Actualizar()` y `Eliminar()` modifican o eliminan registros según el ID. Finalmente, los métodos `TotalSalarios()` y `SalarioMasAlto()` usan LINQ para trabajar con los datos obtenidos: uno suma todos los salarios y el otro encuentra el empleado con el salario más alto. En conjunto, esta clase funciona como la capa que conecta la aplicación con la base de datos.

```
using SistemaEmpleados.Entidad;

namespace SistemaEmpleados.Repositorio;

2 referencias
public interface IEmpleadosRepositorio
{
    4 referencias
    IReadOnlyList<Empleado> ObtenerTodos();
    2 referencias
    int Insertar(Empleado empleado);
    2 referencias
    int Actualizar(Empleado empleado);
    2 referencias
    int Eliminar(int id);
    2 referencias
    decimal TotalSalarios();
    2 referencias
    Empleado? SalarioMasAlto();
}
```

La clase `IEmployadosRepositorio` define un contrato para la gestión de empleados dentro del sistema, estableciendo los métodos necesarios para realizar operaciones básicas sobre la entidad `Empleado`. Entre sus funcionalidades se incluyen obtener la lista de empleados registrados, insertar nuevos empleados, actualizar información existente, eliminar empleados mediante su identificador, calcular el total de salarios y obtener el empleado con el salario más alto. Esta clase forma parte del patrón de diseño Repository, el cual permite separar la lógica de acceso a datos de la lógica principal de la aplicación, mejorando la organización, mantenibilidad y escalabilidad del sistema.

Base de datos

```
1  CREATE DATABASE SistemaEmpleados;
2
3  GO
4
5
6  USE SistemaEmpleados;
7
8  GO
9
10 CREATE TABLE dbo.Empleados
11 (
12     Id INT IDENTITY(1, 1) NOT NULL PRIMARY KEY,
13     Nombre NVARCHAR(200) NOT NULL,
14     Cargo NVARCHAR(200) NOT NULL,
15     Salario DECIMAL(18, 2) NOT NULL
16 );
17 GO
18
```

Bueno creamos una base de datos que se llama SistemaEmpleados dentro de ella creamos la table, que guardaran los siguientes parámetros id, nombre, cargo, salario despues de terminar de crear la base y la tabla solo hacemos la vinculación con el código mediante la clase conexión y el código y la base de datos estaría listo para funcionar.

Ejercicio 5

Control de Inventario

CONTROL DE INVENTARIO

REGISTRAR PRODUCTO

Nombre del producto:

Cantidad:

Categoría:

Precio de compra:

AGREGAR PRODUCTO

OPCIONES

Mostrar todos

Stock menor a 5

Calcular valor total

LISTA DE PRODUCTOS

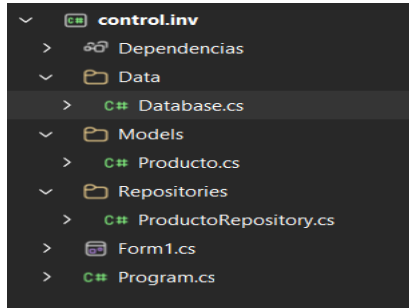
Id	Nombre	Categoría	Cantidad	Precio de Compra	Valor Total
1	caramelo	dulce	8	3.00	24.00
2	tomate	verdura	5	6.00	30.00

Valor total del inventario: (pulse «Calcular valor total»)

Creamos una interfaz grafica de la cual está compuesta de lo siguiente:

- `Panel pnlFooter;`
- `Label lblTotal;`
- `TableLayoutPanel tlpRoot;`
- `Label lblTitulo;`
- `Panel pnlRegistrarOuter;`
- `Label lblHeaderRegistrar;`
- `TableLayoutPanel tlpRegistrar;`
- `Label lblNombre;`
- `TextBox txtNombre;`
- `Label lblCantidad;`
- `TextBox txtCantidad;`
- `Label lblCategoria;`
- `TextBox txtCategoria;`
- `Label lblPrecio;`
- `TextBox txtPrecio;`
- `Button btnAgregar;`
- `Panel pnlOpcionesOuter;`
- `Label lblHeaderOpciones;`
- `FlowLayoutPanel flpOpciones;`
- `Button btnMostrar;`
- `Button btnStockBajo;`
- `Button btnTotal;`
- `Panel pnlListaOuter;`
- `Label lblHeaderLista;`
- `DataGridView dgvProductos;`

Depues creamos 3 carpetas



Cada carpeta tiene su respectiva clase;

La primera carpeta que es data tiene una clase que esta compuesta del siguiente código.

```
1 using Microsoft.Data.SqlClient;
2
3 namespace control.inv.Data;
4
5 1 referencia
6 public static class Database
7 {
8     public const string AppConnectionString =
9         "Server=(localdb)\MSSQLLocalDB;Database=InventarioDB;Trusted_Connection=True;TrustServerCertificate=True;";
10 }
11
```

Esta clase Database sirve para definir y exponer una cadena de conexión constante (AppConnectionString) que apunta a una instancia local de SQL Server LocalDB llamada InventarioDB, usando autenticación de Windows. Esto permite que cualquier otra parte de la aplicación (como repositorios o servicios) pueda acceder a la base de datos de forma centralizada, simplemente referenciando esta constante.

Models, Producto.cs

```
namespace control.inv.Models;
14 referencias
public class Producto
{
    2 referencias
    public int Id { get; set; }
    4 referencias
    public string Nombre { get; set; } = string.Empty;
    4 referencias
    public string Categoria { get; set; } = string.Empty;
    5 referencias
    public int Cantidad { get; set; }
    5 referencias
    public decimal PrecioCompra { get; set; }
    1 referencia
    public decimal ValorTotal => Cantidad * PrecioCompra;
}
```

Este código crea el modelo Producto, que representa la tabla de productos en la base de datos del sistema de inventario. Contiene propiedades como Id, Nombre, Categoria, Cantidad y PrecioCompra para almacenar la información de cada producto. Además, incluye ValorTotal, una propiedad calculada que multiplica la cantidad por el precio de compra para obtener automáticamente el valor total del producto en inventario.

Repositories, ProductoRepository.cs

```
using Microsoft.Data.SqlClient;
using control.inv.Data;
using control.inv.Models;

namespace control.inv.Repositories;

2 referencias
public class ProductoRepository
{
    private readonly string _connectionString = Database.AppConnectionString;

    1 referencia
    public void Insertar(Producto producto)
    {
        const string sql = """
            INSERT INTO dbo.Productos (Nombre, Categoria, Cantidad, PrecioCompra)
            VALUES (@Nombre, @Categoria, @Cantidad, @PrecioCompra);
            """;

        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand(sql, conn);
        cmd.Parameters.AddWithValue("@Nombre", producto.Nombre);
        cmd.Parameters.AddWithValue("@Categoria", producto.Categoria);
        cmd.Parameters.AddWithValue("@Cantidad", producto.Cantidad);
        cmd.Parameters.AddWithValue("@PrecioCompra", producto.PrecioCompra);

        conn.Open();
        cmd.ExecuteNonQuery();
    }

    1 referencia
    public List<Producto> ObtenerTodos() =>
        EjecutarConsulta("SELECT Id, Nombre, Categoria, Cantidad, PrecioCompra FROM dbo.Productos ORDER BY Id;");

    1 referencia
    public List<Producto> ObtenerStockMenorA5() =>
        EjecutarConsulta("SELECT Id, Nombre, Categoria, Cantidad, PrecioCompra FROM dbo.Productos WHERE Cantidad < 5 ORDER BY Cantidad, Id;");

    1 referencia
    public decimal CalcularValorTotalInventario()
    {
        const string sql = """
            SELECT ISNULL(SUM(CAST(Cantidad AS DECIMAL(18,2)) * PrecioCompra), 0)
            FROM dbo.Productos;
            """;

        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand(sql, conn);
        conn.Open();
        var result = cmd.ExecuteScalar();
        return (result is null || result is DBNull) ? 0m : Convert.ToDecimal(result);
    }

    2 referencias
    private List<Producto> EjecutarConsulta(string sql)
    {
        var lista = new List<Producto>();
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand(sql, conn);
        conn.Open();
        using var reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            lista.Add(new Producto
            {
                Id = reader.GetInt32(0),
                Nombre = reader.GetString(1),
                Categoria = reader.GetString(2),
                Cantidad = reader.GetInt32(3),
                PrecioCompra = reader.GetDecimal(4)
            });
        }
        return lista;
    }
}
```

Este código crea la clase **ProductoRepository**, la cual se encarga de manejar todas las operaciones relacionadas con los productos en la base de datos utilizando SQL Server y ADO.NET. Usa la cadena de conexión almacenada en `Database.AppConnectionString` para conectarse a la base de datos. Incluye métodos para insertar nuevos productos (**Insertar**), obtener todos los productos registrados (**ObtenerTodos**), mostrar productos con stock menor a 5 (**ObtenerStockMenorA5**) y calcular el valor total del inventario multiplicando la cantidad por el precio de compra (**CalcularValorTotalInventario**). También contiene el método privado **EjecutarConsulta**, que ejecuta consultas SQL y convierte los resultados obtenidos en objetos de tipo **Producto**.

Clase Program.cs

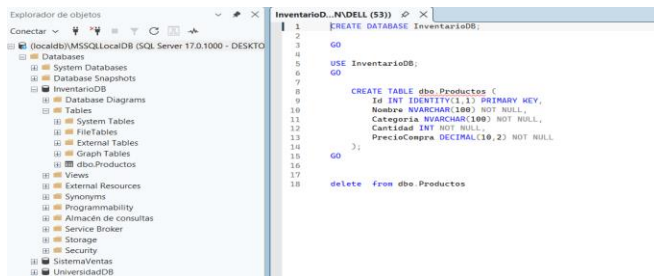
```
1 namespace control.inv;
2 {
3     internal static class Program
4     {
5         [STAThread]
6         private static void Main()
7         {
8             ApplicationConfiguration.Initialize();
9             Application.Run(new Form1());
10        }
11    }
12 }
```

Buscar en Explorador de soluciones (Ctrl+) Q

Solución "control.inv" (1 de 1 proyecto)

- control.inv
 - Dependencias
 - Data
 - Models
 - Repositories
 - Form1.cs
 - Program.cs

Este código corresponde al archivo **Program.cs** y es el punto de inicio de la aplicación Windows Forms. La clase **Program** contiene el método **Main**, que se ejecuta primero al iniciar el sistema. `ApplicationConfiguration.Initialize()` configura la aplicación y prepara el entorno gráfico de Windows Forms, mientras que `Application.Run(new Form1())` abre y ejecuta la ventana principal llamada **Form1**, permitiendo que el usuario interactúe con el sistema.



Creamos la base de datos, el cual esta conformada de la siguiente forma:

Se crea la base de datos

CREATE DATABASE InventarioDB;

Esta instrucción crea una nueva base de datos en SQL Server llamada: InventarioDB , la base de datos será utilizada para guardar toda la información relacionada con el sistema de inventario.

USE InventarioDB;

GO

- USE InventarioDB;

Indica que todas las instrucciones siguientes se ejecutarán dentro de esa base de datos.

- GO

Sirve para separar bloques de ejecución en SQL Server.

```
USE InventarioDB;
GO
```

```
CREATE TABLE dbo.Productos (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Nombre NVARCHAR(100) NOT NULL,
    Categoria NVARCHAR(100) NOT NULL,
    Cantidad INT NOT NULL,
    PrecioCompra DECIMAL(10,2) NOT NULL
);
GO
```

Campo	Tipo de dato	Explicación
Id	INT IDENTITY(1,1)	Identificador único del producto. Se incrementa automáticamente desde 1.
Nombre	NVARCHAR(100)	Guarda el nombre del producto. Máximo 100 caracteres.
Categoria	NVARCHAR(100)	Guarda la categoría del producto.
Cantidad	INT	Almacena la cantidad disponible del producto.
PrecioCompra	DECIMAL(10,2)	Guarda el precio de compra con 2 decimales.

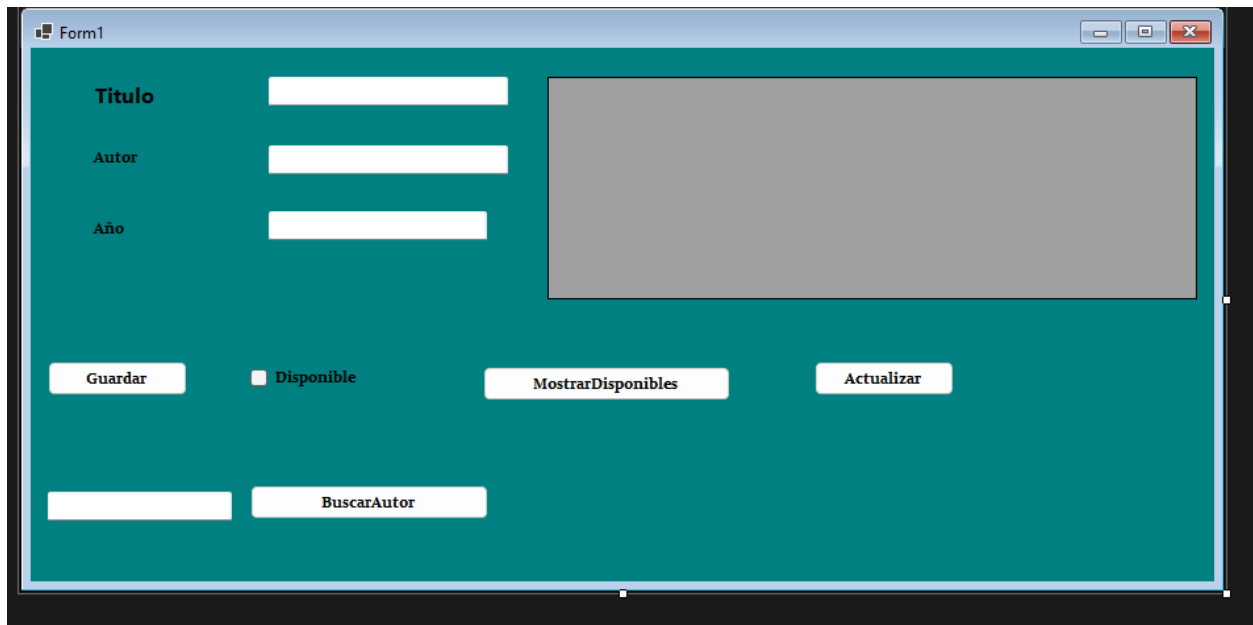
Despues se cierra con un GO y ahí crearíamos nuestra base de datos. Despues hacemos la vinculación

Nombre del servidor

(localdb)\MSSQLLocalDB

Buscamos el nombre de nuestro servidor, lo pegamos en el archivo data en la clase dentro de ella y mediante ese medio copiamos el servidor y tendríamos vinculada nuestra base de datos

Ejercicio 6



Campos de Entrada de Datos (Labels y TextBoxes):

- **Título, Autor y Año:** Son etiquetas que identifican tres cuadros de texto donde el usuario escribe la información básica del libro.
- **CheckBox "Disponible":** Una casilla de verificación para marcar si el libro está en existencia o prestado.

Panel de Visualización (Cuadro Gris):

- Ubicado a la derecha, este espacio (un DataGridView) está destinado a mostrar la lista de libros almacenados en la base de datos de forma tabular.

Botones de Acción Principal:

- **Guardar:** Envía la información escrita en los cuadros de texto a la base de datos para crear un nuevo registro.
- **Actualizar:** Modifica un registro existente con los nuevos datos proporcionados en el formulario.

Funciones de Filtrado y Consulta:

- **MostrarDisponibles:** Al presionarlo, la lista de la derecha se filtra para mostrar únicamente los libros que tienen marcada la casilla de disponibilidad.
- **BuscarAutor:** Funciona junto al cuadro de texto pequeño que tiene a su izquierda; permite localizar libros escritos por un autor específico.

```

namespace ejercicio6
{
    3 references
    public partial class Form1 : Form
    {
        string conexion = "Server=JOSEMPEREZ;Database=EscuelaDB;Trusted_Connection=True;TrustServerCertificate=True;";
        1 reference
        public Form1()
        {
            InitializeComponent();
        }

        1 reference
        private void btnGuardar_Click(object sender, EventArgs e)
        {
            if (txtTitulo.Text == "" || txtAutor.Text == "" || txtAnio.Text == "")
            {
                MessageBox.Show("Todos los campos son obligatorios");
                return;
            }

            if (!int.TryParse(txtAnio.Text, out int anio))
            {
                MessageBox.Show("El año debe ser número");
                return;
            }
        }
    }
}

```

Configuración Global de Conexión:

- Al inicio de la clase, se define la variable `conexion`. Esta contiene los parámetros necesarios para localizar el servidor (JOSEMPEREZ) y la base de datos específica (EscuelaDB).

Constructor del Formulario:

- El método `Form1()` ejecuta `InitializeComponent()`, que es el encargado de "dibujar" todos los botones y cuadros de texto en la pantalla al iniciar el programa.

Evento `btnGuardar_Click` (Validación de texto):

- El programa verifica si los campos de **Título**, **Autor** o **Año** están vacíos (`== ""`).
- Si falta algún dato, muestra el mensaje "Todos los campos son obligatorios" y detiene la ejecución con un `return`.

Validación de Tipo de Dato:

- Utiliza `int.TryParse` para comprobar que el texto ingresado en el campo de **Año** sea realmente un número entero.
- Si el usuario escribió letras (por ejemplo, "dos mil veinte"), el programa muestra el mensaje "El año debe ser número" y cancela la operación.

```

using (SqlConnection con = new SqlConnection(conexion))
{
    string query = "INSERT INTO Libros (Titulo, Autor, AñoPublicacion, Disponible) VALUES (@t, @a, @anio, @disp)";
    SqlCommand cmd = new SqlCommand(query, con);

    cmd.Parameters.AddWithValue("@t", txtTitulo.Text);
    cmd.Parameters.AddWithValue("@a", txtAutor.Text);
    cmd.Parameters.AddWithValue("@anio", anio);
    cmd.Parameters.AddWithValue("@disp", chkDisponible.Checked);

    con.Open();
    cmd.ExecuteNonQuery();
    con.Close();

    MessageBox.Show("Libro guardado");
}
}

1 reference
private void btnMostrarDisponibles_Click(object sender, EventArgs e)
{
    using (SqlConnection con = new SqlConnection(conexion))
    {
        SqlDataAdapter da = new SqlDataAdapter("SELECT * FROM Libros WHERE Disponible = 1", con);
        DataTable dt = new DataTable();
        da.Fill(dt);

        dgvLibros.DataSource = dt;
    }
}

```

Este bloque de código se encarga de tomar la información del formulario y registrarla permanentemente en la base de datos.

- Definición de la Consulta: Se crea una instrucción SQL (INSERT INTO) que indica a qué columnas de la tabla "Libros" (Titulo, Autor, AñoPublicacion, Disponible) se enviarán los datos.
- Uso de Parámetros: Se emplean etiquetas como @t o @anio. Esto protege la aplicación contra ataques de seguridad y asegura que los datos se guarden en el formato correcto.
- Recolección de Datos: El código toma el texto de los campos (txtTitulo.Text), el valor numérico procesado anteriormente (anio) y el estado de la casilla de verificación (chkDisponible.Checked).
- Ejecución:
 - Se abre la conexión (con.Open()).
 - Se ejecuta la orden de guardado (cmd.ExecuteNonQuery()).
 - Se cierra la conexión para liberar memoria.
- Confirmación: Se muestra un mensaje de "Libro guardado" para avisar al usuario que la operación tuvo éxito.

2. Método btnMostrarDisponibles_Click (Filtrado de Existencias)

Este método permite al usuario ver rápidamente qué libros están listos para préstamo o consulta.

- Consulta con Filtro: Utiliza un adaptador de datos para ejecutar una búsqueda específica: `SELECT * FROM Libros WHERE Disponible = 1` Trae toda la información de los libros, pero solo si su estado de disponibilidad es igual a "1" (Verdadero).
- Carga en Memoria: Los resultados se depositan en una tabla temporal llamada `DataTable dt`.
- Actualización de Pantalla: Se asigna esa tabla como origen de datos del control `dgvLibros` (la cuadrícula de la interfaz), refrescando la vista del usuario inmediatamente.

```

1 reference
private void btnBuscarAutor_Click(object sender, EventArgs e)
{
    using (SqlConnection con = new SqlConnection(conexion))
    {
        SqlDataAdapter da = new SqlDataAdapter("SELECT * FROM Libros", con);
        DataTable dt = new DataTable();
        da.Fill(dt);

        var filtrados = dt.AsEnumerable()
            .Where(x => x.Field<string>("Autor")
                .ToLower()
                .Contains(txtBuscarAutor.Text.ToLower()));

        if (filtrados.Any())
            dgvLibros.DataSource = filtrados.CopyToDataTable();
        else
            MessageBox.Show("No se encontraron libros");
    }
}

1 reference
private void btnActualizar_Click(object sender, EventArgs e)
{
    if (dgvLibros.CurrentRow == null)
    {
        MessageBox.Show("Selecciona un libro");
        return;
    }

    int id = Convert.ToInt32(dgvLibros.CurrentRow.Cells["Id"].Value);

    using (SqlConnection con = new SqlConnection(conexion))
    {
        string query = "UPDATE Libros SET Disponible = @disp WHERE Id = @id";
        SqlCommand cmd = new SqlCommand(query, con);
        cmd.Parameters.AddWithValue("@disp", chkDisponible.Checked);
        cmd.Parameters.AddWithValue("@id", id);

        con.Open();
        cmd.ExecuteNonQuery();
        con.Close();

        MessageBox.Show("Disponibilidad actualizada");
    }
}

```

Método `btnBuscarAutor_Click` (Búsqueda Flexible)

Este código permite buscar libros basándose en el nombre del autor, siendo más flexible que una búsqueda exacta.

- Extracción de Datos: Primero, conecta a la base de datos y trae todos los libros de la tabla a un `DataTable`.
- Filtrado con LINQ: Realiza una búsqueda inteligente en la columna "Autor":
 - Convierte todo a minúsculas (`.ToLower()`) para que no importe si el usuario escribe con mayúsculas o minúsculas.
 - Usa `.Contains()`, lo que significa que encontrará resultados aunque solo se escriba una parte del nombre del autor.
- Resultados:
 - Si hay coincidencias, actualiza la cuadrícula (`dgvLibros`) con los libros encontrados.
 - Si no encuentra nada, muestra un mensaje avisando que "No se encontraron libros".

```
1
2 CREATE DATABASE BibliotecaDB;
3 GO
4
5 -
6 USE BibliotecaDB;
7 GO
8
9 -- Crear la tabla principal de libros
10 CREATE TABLE Libros (
11     Id INT IDENTITY(1,1) PRIMARY KEY,
12     Titulo NVARCHAR(200) NOT NULL,
13     Autor NVARCHAR(150) NOT NULL,
14     Año INT NOT NULL,
15     Disponible BIT NOT NULL
16 );
17 GO
18
```

Creamos una base de datos llamada biblioteca el cual le creamos una tabla que guardara los siguientes parámetros como el id, titulo, autor, año y si hay disponible o no depues solo enlazamos la base con el código y tendríamos funcionando nuestro proyecto biblioteca.

Ejercicio 7

Sistema de Ventas

Cliente

Producto

Cantidad

Precio

Total

Calcular Total Guardar Venta Mostrar Total General Mostrar Producto Más Vendido Limpiar

Total general: —

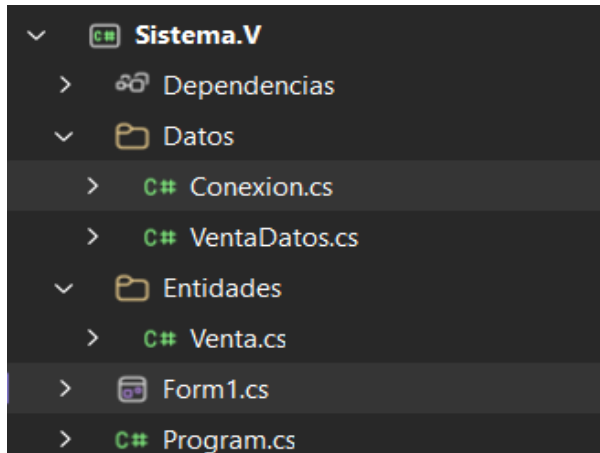
Producto más vendido: —

Nº	Cliente	Producto	Cantidad	Precio	Total
1	Crsitobal	pan	8	9.00	72.00
2	raquel	caramelo	65	3.00	195.00
3	samuel	banano	12	2.00	24.00
4	jared	galleta	2	10.00	20.00

Creamos la interfaz gráfica que está compuesta por los siguientes elementos:

```
Label lblCliente;  
Label lblProducto;  
Label lblCantidad;  
Label lblPrecio;  
Label lblTotal;  
TextBox txtCliente;  
TextBox txtProducto;  
TextBox txtCantidad;  
TextBox txtPrecio;  
TextBox txtTotal;  
Button btnCalcularTotal;  
Button btnGuardarVenta;  
Button btnMostrarTotalGeneral;  
Button btnMostrarProductoMasVendido;  
Button btnLimpiar;  
Label lblTotalGeneral;  
Label lblProductoMasVendido;  
DataGridView dgvVentas;
```

Después de terminar con la interfaz creamos 2 carpetas con sus correspondientes clases:



Datos → Conexion.cs

Se creó la clase para manejar la conexión con la base de datos SQL Server.

Datos → VentaDatos.cs

Se hizo la clase encargada de realizar las operaciones de ventas en la base de datos, como guardar y consultar información.

Entidades → Venta.cs

Se creó el modelo o entidad Venta, donde se almacenan los datos de cada venta.

Form1.cs

Es la interfaz gráfica principal donde el usuario interactúa con el sistema.

Program.cs

Archivo principal que inicia y ejecuta la aplicación Windows Forms.

Datos → Conexion.cs, VentaDatos.cs

```
using System;
using System.Data;
using System.Data.SqlClient;

public class Conexion
{
    private const string NombreBaseDatos = "SistemaVentas";
    private static string _servidorCache;
    private static readonly string[] ServidoresPorDefecto =
    {
        @"(localdb)\MSSQLLocalDB",
        @"SQLEXPRESS",
        @"",
        @"localhost"
    };

    1 referencia
    public string CadenaConexion => ConstruirCadena(ObtenerServidorSql(), NombreBaseDatos);

    1 referencia
    public SqlConnection CrearConexion() => new SqlConnection(CadenaConexion);

    1 referencia
    public void AbrirConexion(SqlConnection conexion)
    {
        if (conexion.State == System.Data.ConnectionState.Closed)
        {
            conexion.Open();
        }
    }

    4 referencia
    public void CerrarConexion(SqlConnection conexion)
    {
        if (conexion.State == System.Data.ConnectionState.Open)
        {
            conexion.Close();
        }
    }

    1 referencia
    public void VerificarConexion()
    {
        using SqlConnection cn = CrearConexion();
        AbrirConexion(cn);
    }

    2 referencia
    private static string ConstruirCadena(string servidor, string baseDatos)
    {
        return @"Data Source={servidor};Initial Catalog={baseDatos};Integrated Security=True;TrustServerCertificate=True;Encrypt=True;";
    }

    1 referencia
    private static string ObtenerServidorSql()
    {
        if (_servidorCache != null)
        {
            return _servidorCache;
        }

        foreach (string candidato in ServidoresPorDefecto)
        {
            if (ProbarConexionMaster(candidato))
            {
                _servidorCache = candidato;
                return _servidorCache;
            }
        }

        throw new InvalidOperationException(
            "No se pudo conectar a SQL Server.\n" +
            "Instale SQL Server a localde y ejecute Sistema V.sql. " +
            "Si es instancia tiene otro nombre, edite ServidoresPorDefecto en Conexion.cs.");
    }

    1 referencia
    private static bool ProbarConexionMaster(string servidor)
    {
        try
        {
            string cs = ConstruirCadena(servidor, "master").Replace("TrustServerCertificate=True;", "TrustServerCertificate=True;Connection Timeout=1;");
            using SqlConnection cn = new SqlConnection(cs);
            cn.Open();
            return true;
        }
        catch
        {
            return false;
        }
    }
}
```

Este código crea la clase Conexion, encargada de manejar toda la conexión entre la aplicación y SQL Server. Define el nombre de la base de datos (SistemaVentas) y varios servidores SQL por defecto que el sistema intentará usar automáticamente, como LocalDB y SQL Express. La propiedad CadenaConexion genera la cadena de conexión necesaria para conectarse a la base de datos, mientras que CrearConexion() crea una nueva conexión SQL. También incluye métodos para abrir (AbrirConexion) y cerrar (CerrarConexion) la conexión de forma segura, además de VerificarConexion() para comprobar si la conexión funciona correctamente. El método ObtenerServidorSql() busca automáticamente un servidor SQL disponible y lo guarda en memoria para reutilizarlo, y ProbarConexionMaster() prueba si el servidor responde correctamente intentando conectarse a la base de datos master. En resumen, esta clase centraliza y automatiza todo el manejo de conexión con SQL Server dentro del sistema de ventas.

```

using System.Data;
using Microsoft.Data.SqlClient;
using Sistema.V.Entidades;

namespace Sistema.V.Datos
{
    [reference]
    public class VentaDatos
    {
        private readonly Conexion _conexion = new Conexion();

        [reference]
        public void Insertar(Venta venta)
        {
            const string sql = @"
INSERT INTO dbo.Ventas (Cliente, Producto, Cantidad, Precio, Total)
VALUES (@Cliente, @Producto, @Cantidad, @Precio, @Total);";

            using SqlConnection cn = _conexion.CrearConexion();
            try
            {
                _conexion.AbrirConexion(cn);
                using SqlCommand cmd = new SqlCommand(sql, cn);
                cmd.Parameters.Add("@Cliente", SqlDbType.NVarChar, 200).Value = venta.Cliente;
                cmd.Parameters.Add("@Producto", SqlDbType.NVarChar, 200).Value = venta.Producto;
                cmd.Parameters.Add("@Cantidad", SqlDbType.Int).Value = venta.Cantidad;

                SqlParameter pPrecio = cmd.Parameters.Add("@Precio", SqlDbType.Decimal);
                pPrecio.Precision = 18;
                pPrecio.Scale = 2;
                pPrecio.Value = venta.Precio;

                SqlParameter pTotal = cmd.Parameters.Add("@Total", SqlDbType.Decimal);
                pTotal.Precision = 18;
                pTotal.Scale = 2;
                pTotal.Value = venta.Total;

                cmd.ExecuteNonQuery();
            }
            finally
            {
                _conexion.CerrarConexion(cn);
            }
        }

        [reference]
        public DataTable ObtenerVentas()
        {
            const string sql = @"
SELECT ROW_NUMBER() OVER (ORDER BY Id) AS [N°],
       Cliente,
       Producto,
       Cantidad,
       Precio,
       Total
FROM dbo.Ventas
ORDER BY Id;";

            DataTable tabla = new DataTable();
            using SqlConnection cn = _conexion.CrearConexion();
            try
            {
                _conexion.AbrirConexion(cn);
                using SqlCommand cmd = new SqlCommand(sql, cn);
                using SqlDataAdapter adaptador = new SqlDataAdapter(cmd);
                adaptador.Fill(tabla);
            }
            finally
            {
                _conexion.CerrarConexion(cn);
            }

            return tabla;
        }
    }
}

```

```

[reference]
public decimal ObtenerTotalGeneral()
{
    const string sql = @"SELECT ISNULL(SUM(Total), 0) FROM dbo.Ventas;";

    using SqlConnection cn = _conexion.CrearConexion();
    try
    {
        _conexion.AbrirConexion(cn);
        using SqlCommand cmd = new SqlCommand(sql, cn);
        object? resultado = cmd.ExecuteScalar();
        return resultado == null || resultado == DBNull.Value
            ? 0m
            : Convert.ToDecimal(resultado);
    }
    finally
    {
        _conexion.CerrarConexion(cn);
    }
}

[reference]
public string ObtenerProductoMasVendido()
{
    const string sql = @"
SELECT TOP 1 Producto
FROM dbo.Ventas
GROUP BY Producto
ORDER BY SUM(Cantidad) DESC;";

    using SqlConnection cn = _conexion.CrearConexion();
    try
    {
        _conexion.AbrirConexion(cn);
        using SqlCommand cmd = new SqlCommand(sql, cn);
        object? resultado = cmd.ExecuteScalar();
        return resultado == null || resultado == DBNull.Value
            ? string.Empty
            : resultado.ToString() ?? string.Empty;
    }
    finally
    {
        _conexion.CerrarConexion(cn);
    }
}

```

Este código crea la clase VentaDatos, encargada de manejar todas las operaciones relacionadas con las ventas en la base de datos usando SQL Server y ADO.NET. Utiliza la clase Conexion para conectarse a la base de datos y contiene métodos para insertar nuevas ventas (Insertar), obtener todas las ventas registradas en una tabla (ObtenerVentas), calcular el total general vendido (ObtenerTotalGeneral) y obtener el producto más vendido (ObtenerProductoMasVendido). También utiliza comandos SQL (SqlCommand), parámetros para enviar datos de forma segura y DataTable para cargar y mostrar información obtenida desde la base de datos.

```

1 namespace Sistema.V.Entidades
2
3 public class Venta
4 {
5     public int Id { get; set; }
6
7     public string Cliente { get; set; } = string.Empty;
8
9     public string Producto { get; set; } = string.Empty;
10
11    public int Cantidad { get; set; }
12
13    public decimal Precio { get; set; }
14
15    public decimal Total { get; set; }
16 }
17

```

Este código crea la clase Venta, la cual representa la entidad o modelo principal de las ventas dentro del sistema. Contiene diferentes propiedades para almacenar la información de cada venta realizada, como Id para identificar el registro, Cliente para guardar el nombre del cliente, Producto para el nombre del producto vendido, Cantidad para la cantidad comprada, Precio para el precio unitario y Total para el monto total de la venta. Esta clase sirve para estructurar y manejar los datos de las ventas, permitiendo enviar y recibir información entre la interfaz gráfica, la lógica del programa y la base de datos de forma organizada.

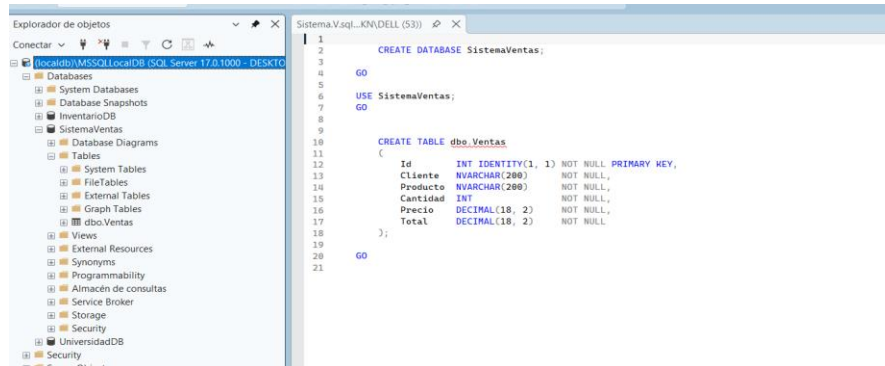
```

1 using Sistema.V.Datos;
2
3 namespace Sistema.V
4 {
5     internal static class Program
6     {
7         [STAThread]
8         static void Main()
9         {
10             ApplicationConfiguration.Initialize();
11
12             try
13             {
14                 new Conexion().VerificarConexion();
15             }
16             catch (Exception ex)
17             {
18                 MessageBox.Show(
19                     "No se pudo conectar a la base SistemaVentas.\n\n" + ex.Message +
20                     "\n\nEjecute Sistema.V.sql en SQL Server (LocalDB o SQLSERVER).",
21                     "Error",
22                     MessageBoxButtons.OK,
23                     MessageBoxIcon.Error);
24                 return;
25             }
26
27             Application.Run(new Form1());
28         }
29     }
30 }

```

Este código corresponde al archivo Program.cs y funciona como el punto de inicio de la aplicación Windows Forms. Dentro del método Main, primero se inicializa la configuración gráfica de la aplicación con ApplicationConfiguration.Initialize(). Luego, el sistema intenta verificar la conexión con la base de datos utilizando la clase Conexion y el método VerificarConexion(). Si ocurre un error al conectarse a SQL Server, se muestra un mensaje de error al usuario mediante MessageBox indicando que debe ejecutar el script Sistema.V.sql en SQL Server. Si la conexión es correcta, la aplicación continúa ejecutando la ventana principal Form1, permitiendo al usuario utilizar el sistema de ventas normalmente.

Base de datos



Creamos la base de datos

CREATE DATABASE SistemaVentas;

Esta instrucción crea una nueva base de datos en SQL Server llamada: SistemaVentas, Esta base de datos servirá para almacenar toda la información relacionada con las ventas de productos.

Selección de la base de datos

USE SistemaVentas;
GO

USE SistemaVentas: Indica que todas las instrucciones SQL siguientes se ejecutarán dentro de la base de datos SistemaVentas.

GO: Se utiliza para separar bloques de instrucciones en SQL Server.

Depues creamos la tabla Ventas

```
CREATE TABLE dbo.Ventas
(  
    Id INT IDENTITY(1, 1) NOT NULL PRIMARY KEY,  
    Cliente NVARCHAR(200) NOT NULL,  
    Producto NVARCHAR(200) NOT NULL,  
    Cantidad INT NOT NULL,  
    Precio DECIMAL(18, 2) NOT NULL,  
    Total DECIMAL(18, 2) NOT NULL  
);
```

Función de la tabla

La tabla Ventas fue creada para registrar las ventas realizadas por los clientes.

Cada registro almacenará:

- El cliente
- El producto vendido
- La cantidad
- El precio
- El total de la venta

Y por ultimos cerramos con GO ara indicar el final del bloque, una vez terminado solo vamos buscamos el nombre de nuestro servidor, y lo ponemos en la clase conexión y listo ya tendríamos nuestra base de datos agarrando los datos del usuario.

Ejercicio 8

Sistema Médico

Este documento explica paso a paso cómo se desarrolló el sistema médico utilizando Windows Forms y SQL Server. El sistema permite registrar pacientes, gestionar información médica y organizar los datos mediante una estructura ordenada basada en programación orientada a objetos.

Paso 1: Creación del Proyecto

Se abrió Visual Studio y se creó un proyecto tipo 'Windows Forms App (.NET Framework)'.

Posteriormente se asignó el nombre correspondiente al sistema médico.

Paso 2: Creación de la Base de Datos

En SQL Server Management Studio se creó la base de datos para almacenar la información de pacientes y consultas.

```
CREATE DATABASE SistemaMedico;
```

```
GO
```

```
USE SistemaMedico;
```

```
CREATE TABLE Pacientes(
```

```
    Id INT PRIMARY KEY IDENTITY(1,1),
```

```
    Nombre NVARCHAR(100),
```

```
    Edad INT,
```

```
    Diagnostico NVARCHAR(200)
```

```
);
```

Paso 3: Organización del Proyecto

El proyecto fue organizado en carpetas para separar responsabilidades y facilitar el mantenimiento del sistema.

■ **Datos**

■ **Entidades**

■ **Repositorios**

■ **Formularios**

Paso 4: Creación de Entidades

Se crearon clases para representar las entidades principales del sistema, como pacientes y consultas médicas.

```
namespace SistemaMedico.Entidades
{
    19 references
    public class Paciente
    {
        3 references
        public int Id { get; set; }

        5 references
        public string Nombre { get; set; }

        5 references
        public int Edad { get; set; }

        5 references
        public string Diagnostico { get; set; }
    }
}
```

Paso 5: Conexión a la Base de Datos

Se creó una clase de conexión para permitir que el sistema pudiera conectarse con SQL Server.

```
namespace SistemaMedico.Datos
{
    using System.Data.SqlClient;

    2 references
    public class ConexionDB
    {
        private string cadena =
            "Server=localhost;Database=SistemaMedicoDB;Trusted_Connection=True;";

        4 references
        public SqlConnection ObtenerConexion()
        {
            return new SqlConnection(cadena);
        }
    }
}
```

Paso 6: Implementación del Repository

Se implementó el patrón Repository para separar la lógica de acceso a datos de la interfaz gráfica.

```
namespace SistemaMedico.Repository
{
    using SistemaMedico.Datos;
    using SistemaMedico.Entidades;
    using System.Collections.Generic;
    using System.Data.SqlClient;

    2 references
    public class PacienteRepository : IPacienteRepository
    {
        ConexionDB conexion = new ConexionDB();

        2 references
        public void Guardar(Paciente paciente)
        {
            using (SqlConnection con = conexion.ObtenerConexion())
            {
                string query =
                    "INSERT INTO Pacientes(Nombre,Edad,Diagnostico) VALUES(@Nombre,@Edad,@Diagnostico)";
                SqlCommand cmd = new SqlCommand(query, con);
            }
        }
    }
}
```

```
        cmd.Parameters.AddWithValue("@Nombre", paciente.Nombre);
        cmd.Parameters.AddWithValue("@Edad", paciente.Edad);
        cmd.Parameters.AddWithValue("@Diagnostico", paciente.Diagnostico);

        con.Open();
        cmd.ExecuteNonQuery();
    }

    2 references
    public List<Paciente> ObtenerTodos()
    {
        List<Paciente> lista = new List<Paciente>();

        using (SqlConnection con = conexion.ObtenerConexion())
        {
            string query = "SELECT * FROM Pacientes";
            SqlCommand cmd = new SqlCommand(query, con);
            con.Open();
        }
    }
}
```

```

        SqlDataReader reader = cmd.ExecuteReader();

        while (reader.Read())
        {
            lista.Add(new Paciente
            {
                Id = (int)reader["Id"],
                Nombre = reader["Nombre"].ToString(),
                Edad = (int)reader["Edad"],
                Diagnostico = reader["Diagnostico"].ToString()
            });
        }

        return lista;
    }

    2 references
    public List<Paciente> BuscarPorNombre(string nombre)
    {

```

```

        Edad = (int)reader["Edad"],
        Diagnostico = reader["Diagnostico"].ToString()
    });
}

return lista;
}

2 references
public List<Paciente> MayoresDe60()
{
    List<Paciente> lista = new List<Paciente>();

    using (SqlConnection con = conexion.ObtenerConexion())
    {
        string query =
            "SELECT * FROM Pacientes WHERE Edad > 60";

        SqlCommand cmd = new SqlCommand(query, con);

        con.Open();

```

```

List<Paciente> lista = new List<Paciente>();

using (SqlConnection con = conexion.ObtenerConexion())
{
    string query =
        "SELECT * FROM Pacientes WHERE Nombre LIKE @Nombre";

    SqlCommand cmd = new SqlCommand(query, con);

    cmd.Parameters.AddWithValue("@Nombre", "%" + nombre + "%");

    con.Open();

    SqlDataReader reader = cmd.ExecuteReader();

    while (reader.Read())
    {
        lista.Add(new Paciente
        {
            Id = (int)reader["Id"],
            Nombre = reader["Nombre"].ToString(),

```

```

        con.Open(),

        SqlDataReader reader = cmd.ExecuteReader();

        while (reader.Read())
        {
            lista.Add(new Paciente
            {
                Id = (int)reader["Id"],
                Nombre = reader["Nombre"].ToString(),
                Edad = (int)reader["Edad"],
                Diagnostico = reader["Diagnostico"].ToString()
            });
        }
    }

    return lista;
}

```

Paso 7: Diseño del Formulario

En el formulario principal se agregaron controles para registrar y visualizar información médica.

Control	Nombre
TextBox	txtNombre
TextBox	txtEdad
TextBox	txtDiagnostico
Button	btnGuardar
Button	btnEliminar
DataGridView	dgvPacientes

Paso 8: Registro de Pacientes

Se programó el botón Guardar para registrar pacientes en la base de datos.

```

private void btnGuardar_Click(object sender, EventArgs e)
{
    if (txtNombre.Text == "" ||
        txtEdad.Text == "" ||
        txtDiagnostico.Text == "")
    {
        MessageBox.Show("Campos vacios");
        return;
    }

    Paciente paciente = new Paciente()
    {
        Nombre = txtNombre.Text,
        Edad = Convert.ToInt32(txtEdad.Text),
        Diagnostico = txtDiagnostico.Text
    };

    repo.Guardar(paciente);
}

```

```

repo.Guardar(paciente);

MostrarPacientes();

MessageBox.Show("Paciente guardado");

```

Paso 9: Visualización de Datos

Los pacientes registrados se muestran automáticamente en un DataGridView.

```

private void MostrarPacientes()
{
    dgvPacientes.DataSource = null;
    dgvPacientes.DataSource = repo.ObtenerTodos();
}

1 reference
private void Form1_Load(object sender, EventArgs e)
{
    MostrarPacientes();
}

```

Paso 10: Mayores de 60

El sistema permite saber cuantos pacientes tiene de 60 en adelante.

```

1 reference
private void btnMayores_Click(object sender, EventArgs e)
{
    dgvPacientes.DataSource = null;

    dgvPacientes.DataSource =
        repo.MayoresDe60();
}

```

Conclusión

El sistema médico desarrollado permite registrar y administrar información de pacientes utilizando Windows Forms y SQL Server. Además, se aplicaron buenas prácticas como el uso de Repository y separación de responsabilidades, logrando un sistema más organizado y fácil de mantener.

Ejercicio 9

Sistema de Facturación con Transacciones

Este documento explica paso a paso cómo se desarrolló el sistema de facturación en Windows Forms utilizando SQL Server, ADO.NET y transacciones SQL. El sistema permite registrar facturas, agregar productos y guardar toda la información de manera segura utilizando COMMIT y ROLLBACK.

Paso 1: Creación del Proyecto

Se abrió Visual Studio y se creó un proyecto tipo 'Windows Forms App (.NET Framework)' llamado SistemaFacturacion.

Paso 2: Creación de la Base de Datos

En SQL Server Management Studio se creó la base de datos llamada FacturacionDB.

```
CREATE DATABASE FacturacionDB;  
GO  
USE FacturacionDB;
```

Después se crearon las tablas Facturas y DetalleFactura.

```
CREATE TABLE Facturas(  
    Id INT PRIMARY KEY IDENTITY(1,1),  
    Cliente NVARCHAR(100),  
    Fecha DATETIME  
);
```

```
CREATE TABLE DetalleFactura(  
    Id INT PRIMARY KEY IDENTITY(1,1),  
    FacturaId INT,  
    Producto NVARCHAR(100),  
    Cantidad INT,  
    Precio DECIMAL(10,2),  
    FOREIGN KEY(FacturaId)  
    REFERENCES Facturas(Id)  
);
```

Paso 3: Organización del Proyecto

El proyecto se organizó utilizando carpetas para separar responsabilidades.

■ Datos

■ Entidades

Paso 4: Creación de Entidades

Se crearon las clases Factura y DetalleFactura dentro de la carpeta Entidades. Estas clases representan las tablas de la base de datos.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace SistemaFacturacion.Entidades
{
    public class Factura
    {
        public int Id { get; set; }

        public string Cliente { get; set; }

        public DateTime Fecha { get; set; }
    }
}

namespace SistemaFacturacion.Entidades
{
    public class DetalleFactura
    {
        public int Id { get; set; }

        public int FacturaId { get; set; }

        public string Producto { get; set; }

        public int Cantidad { get; set; }

        public decimal Precio { get; set; }
    }
}
```

Paso 5: Conexión a SQL Server

En la carpeta Datos se creó la clase ConexionDB.cs para manejar la conexión con SQL Server.

```
namespace SistemaFacturacion.Datos
{
    using System.Data.SqlClient;

    public class ConexionDB
    {
        private string cadena =
            "Server=localhost;Database=FacturacionDB;Trusted_Connection=True;";

        public SqlConnection ObtenerConexion()
        {
            return new SqlConnection(cadena);
        }
    }
}
```

Paso 6: Creación del Repository

Se creó FacturaRepository.cs para manejar todas las operaciones de acceso a datos.

Aquí se implementó la transacción SQL utilizando BeginTransaction(), Commit() y Rollback().

```

public class FacturaRepository
{
    ConexionDB conexion = new ConexionDB();

    1 reference
    public void GuardarFactura(
        Factura factura,
        List<DetalleFactura> detalles)
    {
        SqlConnection con = conexion.ObtenerConexion();

        con.Open();

        SqlTransaction transaccion =
            con.BeginTransaction();
    }

```

```

        factura.Cliente);

        cmdFactura.Parameters.AddWithValue(
            "@Fecha",
            factura.Fecha);

        int facturaId =
            (int)cmdFactura.ExecuteScalar();

        // =====
        // INSERTAR DETALLES
        // =====

        foreach (var detalle in detalles)
        {
            string queryDetalle =
                "INSERT INTO DetalleFactura" +
                "(FacturaId,Producto,Cantidad,Precio)" +
                "VALUES(@FacturaId,@Producto,@Cantidad,@Precio)";

            SqlCommand cmdDetalle =

```

```

                cmdDetalle.ExecuteNonQuery();
            }

            // =====
            // TODO SALIÓ BIEN
            // =====

            transaccion.Commit();

            MessageBox.Show(
                "Factura guardada");
        }
        catch (Exception ex)
        {
            // =====
            // HUBO ERROR
            // =====

            transaccion.Rollback();

            MessageBox.Show(

```

```

try
{
    // =====
    // INSERTAR FACTURA
    // =====

    string queryFactura =
        "INSERT INTO Facturas(Cliente,Fecha) " +
        "OUTPUT INSERTED.Id " +
        "VALUES(@Cliente,@Fecha)";

    SqlCommand cmdFactura =
        new SqlCommand(
            queryFactura,
            con,
            transaccion);

    cmdFactura.Parameters.AddWithValue(
        "@Cliente",

```

```

        new SqlCommand(
            queryDetalle,
            con,
            transaccion);

    cmdDetalle.Parameters.AddWithValue(
        "@FacturaId",
        facturaId);

    cmdDetalle.Parameters.AddWithValue(
        "@Producto",
        detalle.Producto);

    cmdDetalle.Parameters.AddWithValue(
        "@Cantidad",
        detalle.Cantidad);

    cmdDetalle.Parameters.AddWithValue(
        "@Precio",
        detalle.Precio);

```

```

        transaccion.Rollback();

        MessageBox.Show(
            "Error: " + ex.Message);
    }
    finally
    {
        con.Close();
    }
}

```

La transacción garantiza que toda la información se guarde correctamente. Si ocurre un error, se cancelan todos los cambios.

Paso 7: Diseño del Formulario

En el formulario principal se agregaron controles para ingresar información.

Control	Nombre
TextBox	txtCliente
TextBox	txtProducto
TextBox	txtCantidad
TextBox	txtPrecio
Button	btnAgregar
Button	btnGuardarFactura
DataGridView	dgvDetalles

Paso 8: Variables Globales del Formulario

Dentro de Form1.cs se declararon variables globales para poder utilizar el repositorio y almacenar temporalmente los detalles.

```
FacturaRepository repo = new FacturaRepository();
```

```
List<DetalleFactura> listaDetalles =  
new List<DetalleFactura>();
```

Paso 9: Agregar Productos

El botón Agregar permite ingresar productos al DataGridView antes de guardar la factura.

```
private void btnAgregar_Click(  
    object sender,  
    EventArgs e)  
{  
    DetalleFactura detalle =  
        new DetalleFactura()  
    {  
        Producto = txtProducto.Text,  
        Cantidad = Convert.ToInt32(  
            txtCantidad.Text),  
        Precio = Convert.ToDecimal(  
            txtPrecio.Text)  
    };  
    listaDetalles.Add(detalle);  
    dgvDetalles.DataSource = null;  
    dgvDetalles.DataSource = listaDetalles;  
}
```

Paso 10: Guardar Factura

Cuando el usuario presiona el botón Guardar Factura, se crea un objeto Factura y se envía junto a la lista de detalles al Repository.

```
1 reference
private void btnGuardarFactura_Click(
object sender,
EventArgs e)
{
    Factura factura =
        new Factura()
        {
            Cliente = txtCliente.Text,
            Fecha = DateTime.Now
        };

    repo.GuardarFactura(
        factura,
        listaDetalles);
}
```

Conclusión

El sistema desarrollado permite registrar facturas y detalles utilizando Windows Forms y SQL Server. Además, se implementó el uso de transacciones para garantizar la integridad de los datos. Gracias al patrón Repository, la lógica de acceso a datos quedó separada de la interfaz gráfica, mejorando la organización y mantenimiento del proyecto.

Ejercicio 10

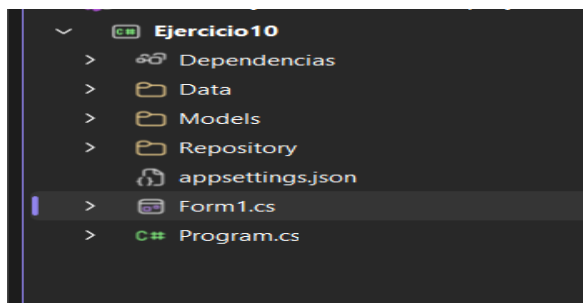
Sistema universitario

Primero creamos la interfaz grafica

Id	Nombre	Edad	Nombre carrera
1	cristobal	19	sistema
2	raquel	19	electronica
3	samuel	24	quimica

La cual esta conformada por

```
GroupBox grpCarreras;  
Label lblNombreCarrera;  
TextBox txtNombreCarrera;  
Button btnRegistrarCarrera;  
  
GroupBox grpEstudiantes;  
Label lblNombreEstudiante;  
TextBox txtNombreEstudiante;  
Label lblEdad;  
NumericUpDown nudEdad;  
Label lblCarreraEstudiante;  
ComboBox cmbCarreraEstudiante;  
Button btnRegistrarEstudiante;  
  
Label lblFiltro;  
ComboBox cmbFiltrarCarrera;  
DataGridView dgvEstudiantes;
```



Creamos su carpeta con su respectiva clase de cada una y un archivo que está conformado de la siguiente manera.

Data → UniversidadContext.cs

Se hizo la conexión y configuración de la base de datos.

Models → Carrera.cs y Estudiante.cs

Se crearon las clases que representan las tablas/modelos del sistema.

Repository

Se hizo la capa que maneja las operaciones de base de datos (guardar, editar, eliminar y buscar).

ICarreraRepository.cs e IEstudianteRepository.cs

Son interfaces para definir los métodos que deben implementar los repositorios.

CarreraRepository.cs y EstudianteRepository.cs

Implementan la lógica para trabajar con la base de datos.

appsettings.json

Se usa para guardar configuraciones, como la cadena de conexión SQL.

Form1.cs

Es la interfaz gráfica principal del programa.

Program.cs

Archivo que inicia la aplicación.

Explicación de los códigos

Data → UniversidadContext.cs

```
1  using Microsoft.EntityFrameworkCore;
2  using Ejercicio10.Models;
3
4  namespace Ejercicio10.Data;
5
6  9 referencias
7  public class UniversidadContext : DbContext
8  {
9      1 referencia
10     public UniversidadContext(DbContextOptions<UniversidadContext> options)
11         : base(options)
12     {
13
14         3 referencias
15         public DbSet<Carrera> Carreras => Set<Carrera>();
16
17         3 referencias
18         public DbSet<Estudiante> Estudiantes => Set<Estudiante>();
19
20         0 referencias
21         protected override void OnModelCreating(ModelBuilder modelBuilder)
22         {
23             modelBuilder.Entity<Estudiante>()
24                 .HasOne(e => e.Carrera)
25                 .WithMany(c => c.Estudiantes)
26                 .HasForeignKey(e => e.CarreraId);
27         }
28     }
```

Este código crea el contexto de la base de datos usando Entity Framework Core y sirve para conectar la aplicación con SQL. Define las tablas Carreras y Estudiantes mediante DbSet, permitiendo guardar, buscar, editar y eliminar datos. Además, configura la relación entre ambas tablas, indicando que un estudiante pertenece a una carrera y que una carrera puede tener muchos estudiantes, utilizando CarreraId como llave foránea.

Models → Carrera.cs y Estudiante.cs.

```
using System.Collections.Generic;

namespace Ejercicio10.Models;

14 referencias
public class Carrera
{
    2 referencias
    public int Id { get; set; }
    4 referencias
    public string NombreCarrera { get; set; } = string.Empty;

    1 referencia
    public ICollection<Estudiante> Estudiantes { get; set; } = new List<Estudiante>();
}
```

Este código crea el modelo Carrera, que representa la tabla de carreras en la base de datos. Contiene un Id como identificador principal y NombreCarrera para guardar el nombre de la carrera. Además, tiene una colección de Estudiantes que permite relacionar varios estudiantes con una sola carrera, estableciendo una relación de uno a muchos.

```
namespace Ejercicio10.Models;

18 referencias
public class Estudiante
{
    2 referencias
    public int Id { get; set; }
    3 referencias
    public string Nombre { get; set; } = string.Empty;
    3 referencias
    public int Edad { get; set; }

    3 referencias
    public int CarreraId { get; set; }
    4 referencias
    public Carrera? Carrera { get; set; }
}
```

Este código crea el modelo Estudiante, que representa la tabla de estudiantes en la base de datos. Contiene un Id como identificador principal, Nombre para guardar el nombre del estudiante y Edad para almacenar su edad. Además, incluye CarreraId como llave foránea y la propiedad Carrera, que permite relacionar cada estudiante con una carrera específica.

Repository

1-2 ICarreraRepository.cs e IEstudianteRepository.cs

3-4 CarreraRepository.cs y EstudianteRepository.cs

```
using Ejercicio10.Models;

namespace Ejercicio10.Repository;

2 referencias
public interface ICarreraRepository
{
    2 referencias
    Task<List<Carrera>> ObtenerTodas();
    1 referencia
    Task<Carrera?> ObtenerPorId(int id);
    2 referencias
    Task<Carrera> Crear(Carrera carrera);
}
```

Este código crea una interfaz llamada ICarreraRepository, la cual define los métodos que debe tener el repositorio de carreras para trabajar con la base de datos. Incluye métodos para obtener todas las carreras, buscar una carrera por su ID y crear una nueva carrera. Sirve para organizar y separar la lógica de acceso a datos del resto de la aplicación.

```
using Ejercicio10.Models;

namespace Ejercicio10.Repository;

2 referencias
public interface IEstudianteRepository
{
    2 referencias
    Task<List<Estudiante>> ObtenerTodos();
    2 referencias
    Task<List<Estudiante>> FiltrarPorCarrera(int carreraId);
    2 referencias
    Task<Estudiante> Crear(Estudiante estudiante);
}
```

Este código crea una interfaz llamada `IEstudianteRepository`, la cual define los métodos que debe implementar el repositorio de estudiantes para manejar la base de datos. Incluye funciones para obtener todos los estudiantes, filtrar estudiantes por carrera y crear nuevos estudiantes. Sirve para mantener organizada la lógica de acceso a datos dentro de la aplicación.

```
using Microsoft.EntityFrameworkCore;
using Ejercicio10.Data;
using Ejercicio10.Models;

namespace Ejercicio10.Repository;

2 referencias
public class EstudianteRepository : IEstudianteRepository
{
    private readonly UniversidadContext _context;

    1 referencia
    public EstudianteRepository(UniversidadContext context)
    {
        _context = context;
    }

    2 referencias
    public async Task<List<Estudiante>> ObtenerTodos()
    {
        return await _context.Estudiantes
            .AsNoTracking()
            .Include(e => e.Carrera)
            .ToListAsync();
    }

    2 referencias
    public async Task<List<Estudiante>> FiltrarPorCarrera(int carreraId)
    {
        return await _context.Estudiantes
            .AsNoTracking()
            .Include(e => e.Carrera)
            .Where(e => e.CarreraId == carreraId)
            .ToListAsync();
    }

    2 referencias
    public async Task<Estudiante> Crear(Estudiante estudiante)
    {
        _context.Estudiantes.Add(estudiante);
        await _context.SaveChangesAsync();
        return estudiante;
    }
}
```

Este código crea la clase `EstudianteRepository`, encargada de manejar las operaciones de la tabla `Estudiantes` en la base de datos usando Entity Framework. Implementa la interfaz `IEstudianteRepository` y permite obtener todos los estudiantes junto con su carrera, filtrar estudiantes por carrera y registrar nuevos estudiantes. También utiliza `UniversidadContext` para conectarse y guardar los datos en la base de datos.

```
1 using Microsoft.EntityFrameworkCore;
2 using Ejercicio10.Data;
3 using Ejercicio10.Models;
4
5 namespace Ejercicio10.Repository;
6
7 2 referencias
8 public class CarreraRepository : ICarreraRepository
9 {
10     private readonly UniversidadContext _context;
11
12     1 referencia
13     public CarreraRepository(UniversidadContext context)
14     {
15         _context = context;
16     }
17
18     2 referencias
19     public async Task<List<Carrera>> ObtenerTodas()
20     {
21         return await _context.Carreras.AsNoTracking().ToListAsync();
22     }
23
24     1 referencia
25     public async Task<Carrera?> ObtenerPorId(int id)
26     {
27         return await _context.Carreras.FindAsync(id);
28     }
29
30     2 referencias
31     public async Task<Carrera> Crear(Carrera carrera)
32     {
33         _context.Carreras.Add(carrera);
34         await _context.SaveChangesAsync();
35         return carrera;
36     }
37 }
```

Este código crea la clase `CarreraRepository`, encargada de manejar las operaciones de la tabla `Carreras` en la base de datos utilizando Entity Framework. Implementa la interfaz `ICarreraRepository` y permite obtener todas las carreras, buscar una carrera por su ID y registrar nuevas carreras. Además, usa `UniversidadContext` para conectarse y guardar la información en la base de datos.

```

{
  "ConnectionStrings": {
    "ConexionSQL": "Server=(localdb)\\MSSQLLocalDB;Database=UniversidadDB;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}

```

El archivo appsettings.json es un archivo de configuración del proyecto. Sirve para guardar datos importantes que la aplicación necesita, como la conexión a la base de datos. En este caso, dentro de ConnectionStrings se creó una cadena llamada ConexionSQL, la cual indica que el sistema se conectará a una base de datos SQL Server LocalDB llamada UniversidadDB. También incluye configuraciones de seguridad y confianza del servidor para permitir la conexión correctamente.

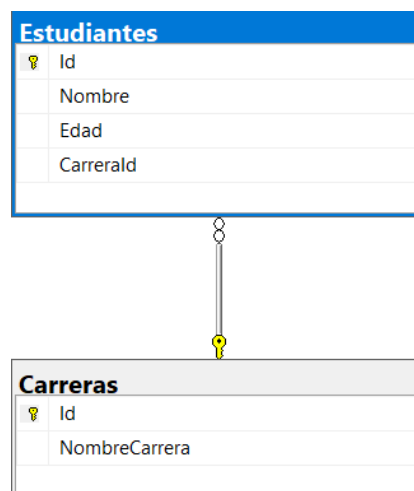
Modelo de datos

La base de datos tiene dos tablas con una relación:

Tabla Campos

Carreras	Id (llave primaria), NombreCarrera
Estudiantes	Id, Nombre, Edad, CarreraId (llave foránea)

Estudiantes.CarreraId apunta a Carreras.Id. Así cada estudiante pertenece a una carrera.



Paso 1 – Crear la base de datos

Paso 2 – Registrar una carrera

- El usuario escribe el nombre de la carrera en el campo de texto.
- Ejemplo: escribe "Derecho".

Ejercicio 10 — Sistema universitario

Registrar carrera

Nombre carrera:

Registrar estudiante

Nombre: Edad:

Carrera:

Filtrar carrera:

Id	Nombre	Edad	Nombre carrera
1	Megumi Fushiguro	18	Ing. Sistemas

```
CREATE DATABASE UniversidadDB;
GO

USE UniversidadDB;
GO

CREATE TABLE Carreras (
    Id INT PRIMARY KEY IDENTITY(1,1),
    NombreCarrera NVARCHAR(100) NOT NULL
);

CREATE TABLE Estudiantes (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Nombre NVARCHAR(100) NOT NULL,
    Edad INT NOT NULL,
    CarreraId INT NOT NULL,

    CONSTRAINT FK_Estudiantes_Carreras
    FOREIGN KEY (CarreraId)
    REFERENCES Carreras(Id)
);
```

Para hacer las tablas relacionales es la misma lógica, cuando terminemos de crear una tabla solo venimos y damos crear una nueva tabla que trata del mismo campus en la misma base de datos y ya, ahora hay dos tablas en una base de datos que trabajan en conjunto para dar una sola información al usuario.

- Luego hace clic en el botón "Registrar carrera".

- El sistema guarda automáticamente la carrera en la tabla Carreras con un Id único.
- La carrera ya está disponible para ser asignada a estudiantes.

Carrera: Ing. Sistemas Registrar estudiante

Filtrar carrera: Todas las carreras

Id	Nombre	Edad	Nombre carrera
1	Ing. Sistemas	18	Ing. Sistemas

Paso 3 – Registrar un estudiante

- El usuario ingresa el nombre del estudiante. Ejemplo: *Orlando Perez*.
- Luego ingresa la edad. Ejemplo: **21**.
- Después selecciona una carrera del menú desplegable. Ejemplo: *Medicina*.
- Finalmente hace clic en el botón "Registrar estudiante".

Ejercicio 10 — Sistema universitario

Registrar carrera

Nombre carrera: Registrar carrera

Registrar estudiante

Nombre: Orlando Perez Edad: 21

Carrera: Medicina Registrar estudiante

Filtrar carrera: Todas las carreras

Id	Nombre	Edad	Nombre carrera
1	Megumi Fushiguro	18	Ing. Sistemas
2	Maria Fernanda	20	Derecho

- El sistema obtiene el Id de la carrera seleccionada y lo guarda en el campo Carrerald de la tabla Estudiantes.

- El estudiante queda automáticamente vinculado a su carrera.

Filtrar carrera: Todas las carreras ▼			
Id	Nombre	Edad	Nombre carrera
1	Megumi Fushiguro	18	Ing. Sistemas
2	María Fernanda	20	Derecho
3	Orlando Perez	21	Medicina